On the Efficient Discovery of Maximum k-Defective Biclique

Donghang Cui
Beijing Institute of Technology
cuidonghang@bit.edu.cn

Ronghua Li
Beijing Institute of Technology
lironghuabit@126.com

Qiangqiang Dai
Beijing Institute of Technology
qiangd66@gmail.com

Hongchao Qin
Beijing Institute of Technology
qhc.neu@gmail.com

Guoren Wang
Beijing Institute of Technology
wanggrbit@gmail.com

Abstract

The problem of identifying the maximum edge biclique in bipartite graphs has attracted considerable attention in bipartite graph analysis, with numerous real-world applications such as fraud detection, community detection, and online recommendation systems. However, real-world graphs may contain noise or incomplete information, leading to overly restrictive conditions when employing the biclique model. To mitigate this, we focus on a new relaxed subgraph model, called the k-defective biclique, which allows for up to k missing edges compared to the biclique model. We investigate the problem of finding the maximum edge k-defective biclique in a bipartite graph, and prove that the problem is NP-hard. To tackle this computation challenge, we propose a novel algorithm based on a new branch-and-bound framework, which achieves a worst-case time complexity of $O(m\alpha_k^n)$, where $\alpha_k < 2$. We further enhance this framework by incorporating a novel pivoting technique, reducing the worst-case time complexity to $O(m\beta_k^n)$, where $\beta_k < \alpha_k$. To improve the efficiency, we develop a series of optimization techniques, including graph reduction methods, novel upper bounds, and a heuristic approach. Extensive experiments on 10 large real-world datasets validate the efficiency and effectiveness of the proposed approaches. The results indicate that our algorithms consistently outperform state-of-the-art algorithms, offering up to 1000× speedups across various parameter settings.

1 Introduction

Bipartite graph is a commonly encountered graph structure consisting of two disjoint vertex sets and an edge set, where each edge connects vertices from the two different sets. In recent decades, bipartite graphs have been utilized as a fundamental tool for modeling various binary relationships, such as user-product interactions in e-commerce [31], authorship connections between authors and publications [30], and associations between genes and phenotypes in bioinformatics [18, 39]. Mining cohesive subgraphs from bipartite graphs has proven to be critically important across diverse domains [16, 46]. For instance, identifying cohesive subgraphs in reviewer-product graphs facilitates the identification of fraudulent reviews or camouflage attacks [25].

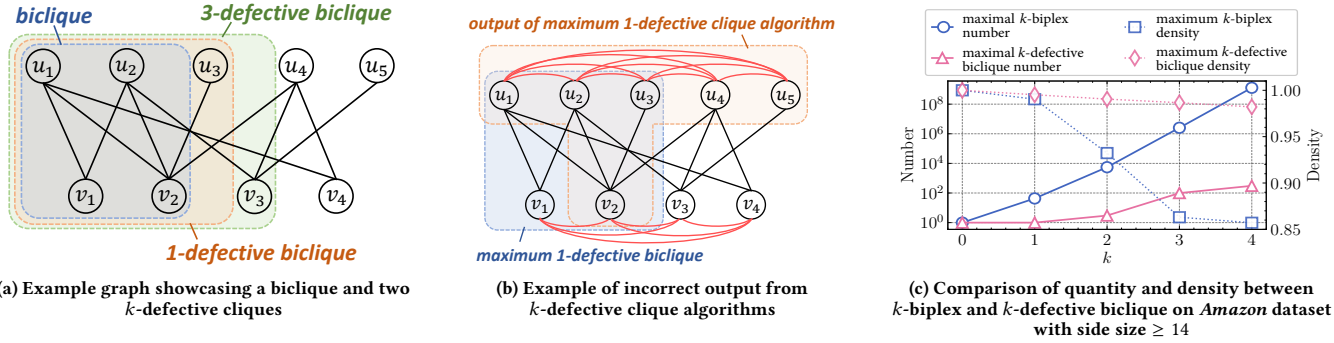
The biclique, which connects all vertices on both sides, serves as a fundamental cohesive subgraph structure [32, 37, 47] due to its wide applications [4, 6, 28, 29]. Extensive methods for maximum biclique search have been developed in recent years [4, 16, 19, 23, 34, 36]. However, real-world graphs often contain noisy or incomplete information [33], making the biclique model overly restrictive for capturing cohesive subgraphs on such a kind of bipartite graphs. More relaxed biclique models offer a promising alternative to address this challenge. The k-biplex model, which allows each vertex to have at most k non-neighbors, has recently attracted significant attention [16, 50, 51]. However, due to its excessive structural relaxation, a slightly increase in k can lead to: (1) a sharp rise in quantity, potentially reducing the mining efficiency; and (2) a significant decrease in density, undermining the capability to represent cohesive communities.

Inspired by the widely studied k-defective clique in traditional graphs [12, 13, 49], we introduce a novel cohesive subgraph model for bipartite graphs, named k-defective biclique. Specifically, a k-defective biclique is a subgraph of a bipartite graph containing at most k missing edges. Fig. 1a illustrates an example graph showcasing two k-defective bicliques, where the 1-defective biclique has one missing edge (u_3, v_1), and the 3-defective biclique has three missing edges (u_1, v_3), (u_3, v_1) and (u_3, v_3).

The k-defective biclique offers greater flexibility over biclique, as its density can be controlled by adjusting k to suit specific requirements. When $k = 0$, the k-defective biclique degenerates to the biclique, thereby illustrating its generalization capability. In comparison to the k-biplex, k-defective biclique offers a more fine-grained relaxation of the biclique due to its stricter retention of edges. As illustrated in Fig. 1c, the k-defective biclique exhibits much closer quantity and density to the biclique compared to k-biplex, especially for larger k. Here, we refer to a k-defective biclique as "maximal" if it cannot be contained within any other k-defective biclique (or k-biplex), and as "maximum" when it is the maximal k-defective biclique (or k-biplex) with the largest number of edges. In this paper, we focus mainly on the problem of identifying a maximum k-defective biclique. We prove that the maximum k-defective biclique search problem is NP-hard, indicating that no polynomial-time algorithms exist for solving this problem unless NP=P.

Notably, adapting existing maximum/maximal k-defective clique algorithms [9, 13] and maximum biclique algorithms [23, 36] to our problem poses significant challenges for the following reasons: (1) Adapting the maximum k-defective clique algorithm involves fully connecting vertices on each side of the bipartite graph (as shown in Fig. 1b). However, applying such algorithms to these graphs may yield incorrect results. For instance, in Fig. 1b, the maximum 1-defective biclique has 5 edges whereas the output of maximum 1-defective clique algorithms yields only 4 edges; (2) Applying maximal k-defective clique algorithms on such each-side-fully-connected graphs will generate a large number of redundant branches and is computationally expensive (as evidenced in Table 2); and (3) Adapting maximum biclique algorithms to our problem involves a preprocess of adding k edges on the original graph, which generates in $\binom{\overline{m}}{k}$ possible input graphs where $\overline{m}$ denotes the number of missing edges. Searching for the maximum biclique across all such input graphs remains highly inefficient, making this approach impractical for large-scale graphs.

Contributions. To efficiently address the maximum k-defective biclique search problem, we conduct an in-depth study of its properties and develop two novel algorithms with the worst-case time complexity less than $O^*(2^n)$. We also present a series of optimization techniques to further improve the efficiency of our algorithms. Finally, we evaluate the effectiveness and efficiency of our methods



(a) Example graph showcasing a biclique and two k-defective cliques

(b) Example of incorrect output from k-defective clique algorithms

(c) Comparison of quantity and density between k-biplex and k-defective biclique on *Amazon* dataset with side size ≥ 14

Figure 1: Illustrations of k-defective biclique: examples and comparisons

through comprehensive experiments. The main contributions of this paper are summarized as follows:

A new cohesive subgraph model. We propose a new cohesive subgraph model for bipartite graphs named the k-defective biclique, which is defined as a subgraph with at most k missing edges. We then demonstrate that the problem of finding the maximum k-defective biclique in bipartite graphs is NP-hard.

Two Novel Search Algorithms. We propose two novel maximum k-defective biclique search algorithms with nontrivial worst-case time complexity guarantees. The first algorithm utilizes a branch-and-bound technique combined with a newly-developed binary branching strategy, which has a time complexity of $O(m\alpha_k^n)$, where n and m represent the number of vertices and edges of the bipartite graph respectively, and α_k is a positive real number strictly less than 2 (e.g., for $k = 1, 2, 3$, $\alpha_k = 1.911, 1.979$, and 1.995, respectively). The second algorithm employs an innovative pivoting-based branching strategy that effectively integrates with our binary branching strategy, which achieves a lower time complexity of $O(m\beta_k^n)$, where β_k is a positive real number strictly less than α_k. (e.g., when $k = 1, 2, 3$, $\beta_k = 1.717, 1.856$, and 1.931, respectively).

Nontrivial optimization techniques. We also develop a series of nontrivial optimization techniques to improve the efficiency of our proposed algorithms, including graph reduction techniques, novel upper-bound techniques, and a heuristic approach. Specifically, the graph reduction techniques focus on reducing the size of the input bipartite graph based on constraints, such as the core and the number of common neighbors. We devise tight vertex and edge upper bounds to minimize unnecessary branching, and then design an efficient algorithm to compute the proposed upper bounds with linear time complexity. Additionally, we propose a heuristic approach by incrementally expanding subsets in a specific order to obtain an initial large k-defective biclique, which helps filter out smaller k-defective bicliques from further computation.

Comprehensive experimental studies. We conduct extensive experiments to evaluate the efficiency, effectiveness, and scalability of our algorithms on 11 large real-world graphs. The experimental results show that our algorithms consistently outperform the state-of-the-art baseline method [15] across all parameter settings. In particular, our algorithm achieves improvements of up to three orders of magnitude on most datasets. For instance, on the *Twitter* (9M vertices, 10M edges) dataset with $k = 2$, our algorithm takes 10 seconds while the baseline method cannot finish within 3 hours. Additionally, we perform a case study on a fraud detection task to show the high effectiveness of our solutions in real-world applications.

2 Preliminaries

Let $G = (U, V, E)$ be an undirected bipartite graph, where U and V are two disjoint sets of vertices, and $E \subseteq U \times V$ is the set of edges. Denote by $n = |U| + |V|$ and $m = |E|$ the total number of vertices and edges in G, respectively. Denote by $S = (U_S, V_S)$ a vertex subset of G, where $U_S \subseteq U$ and $V_S \subseteq V$. Let $v \in S$ indicate that $v \in U_S \cup V_S$. Let $G(S) = (U_S, V_S, E_S)$ be the subgraph of G induced by S, where $E_S = \{(u, v) \mid u \in U_S \wedge v \in V_S \wedge (u, v) \in E\}$. For $u \in U$, denote by $N(u) = \{v \in V \mid (u, v) \in E\}$ and $\overline{N}(u) = \{v \in V \mid (u, v) \notin E\}$ the set of neighbors and non-neighbors of u in G, respectively. Similar definitions apply for the vertices in U. Let $d(u) = |N(u)|$ and $\overline{d}(u) = |\overline{N}(u)|$ be the degree and non-degree of u in G, respectively. For a vertex subset (or a subgraph) S of G, let $N_S(u) = N(u) \cap (U_S \cup V_S)$ and $\overline{N}_S(u) = \{v \in S \setminus N(u) \mid u \text{ and } v \text{ are on opposite sides}\}$ be the set of u's neighbors and non-neighbors in S. respectively. Let $d_S(u) = |N_S(u)|$ and $\overline{d}_S(u) = |\overline{N}_S(u)|$ be the degree and non-degree of u in S, respectively. For a bipartite graph $G = (U, V, E)$, we refer to $(u, v) \in U \times V$ as a non-edge of G if $(u, v) \notin E$. We denote $\overline{E}_S$ the set of non-edges in $G(S)$, i.e., $\overline{E}_S = U_S \times V_S \setminus E_S$. The formal definition of k-defective biclique is given below.

Definition 1 (k-defective biclique). Given a bipartite graph $G = (U, V, E)$, a k-defective biclique $D = (U_D, V_D, E_D)$ of G is a subgraph of G that has at most k non-edges, i.e., $|U_D \times V_D \setminus E_D| \leq k$.

The k-defective biclique corresponds to the biclique if $k = 0$, which indicates that k-defective biclique represents a more general structure. If a k-defective biclique of G contains the most number of edges among all k-defective bicliques in G, we refer to it as a maximum k-defective biclique in G. To facilitate the illustration, we denote the maximum k-defective biclique as k-MDB.

In this paper, we focus on the problem of searching for a k-MDB in a bipartite graph. However, we observe that a k-MDBs in many real-world graphs may exhibit a skewed structure, similar to that of maximum bicliques, where one side has a large number of vertices while the other side may contain only a few (or even one) vertices. Such k-MDBs often hold limited practical values in network analysis. To address this issue, we define a size threshold θ, representing the minimum number of vertices required on both sides of the k-MDB. Furthermore, we impose the condition $\theta > k$ to ensure that the k-MDB is connected, as demonstrated in the following lemma.

Lemma 1. Given a bipartite graph $G = (U, V, E)$ and a k-defective biclique $D = (U_D, V_D, E_D)$ of G, if $|U_D| > k$ and $|V_D| > k$, we establish that D is a connected subgraph in G.

Proof. Assume that a k-defective biclique D is not connected when $|U_D| > k$ and $|V_D| > k$. In this case, D can be decomposed into two nonempty subgraphs $D_1 = (U_{D_1}, V_{D_1}, E_{D_1})$ and $D_2 = (U_{D_2}, V_{D_2}, E_{D_2})$, with the results of $E_{D_1} \cup E_{D_2} = E_D$.The number of non-edges between D_1 and D_2 is given by $|U_{D_1}| \cdot |V_{D_2}| + |U_{D_2}| \cdot |V_{D_1}|$, which exceeds $|U_D|$ (or $|V_D|$) and is thus larger than k. This contradicts the assumption that D is a k-defective biclique. Therefore, the proof is complete. □

Based on Lemma 1, we formally define the problem addressed in this paper as follows.

Problem statement (MDB search problem)**.** Given a bipartite graph $G = (U, V, E)$ and two integers k and θ with $\theta > k$, the MDB search problem aims to find a connected k-MDB $D = (U_D, V_D, E_D)$ of G satisfying $|U_D| \geq \theta$ and $|V_D| \geq \theta$.

Theorem 2. The MDB search problem is NP-hard.

Proof. We establish the NP-hardness of the MDB search problem through a polynomial-time reduction from the classical NP-complete maximum clique search problem. Due to space limitation, we present the complete version of this proof and all the omitted proofs of this paper in [1]. □

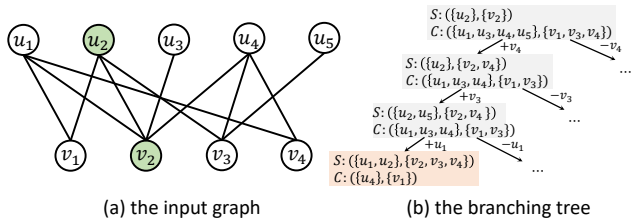
3 Our Proposed Algorithms

In this section, we propose two novel algorithms to solve the k-MDB search problem. Specifically, we first propose a new branch-and-bound algorithm that utilizes a newly-designed binary branching strategy, achieving the worst-case time complexity of $O(m\alpha_k^n)$, where $\alpha_k < 2$ (e.g. $k = 1, 2$, and 3, we have $\alpha_k = 1.911, 1.979$, and 1.995, respectively). To further improve the efficiency, we propose a novel pivoting-based algorithm that incorporates a well-designed pivoting-based branching strategy with our proposed binary branching strategy. We prove that the time complexity of the improved algorithm is bounded by $O(m\beta_k^n)$, where $\beta_k < \alpha_k$ (e.g., when $k = 1, 2, 3$, $\beta_k = 1.717, 1.856$, and 1.931, respectively). To the best of our knowledge, the algorithms we proposed in this section are the first two k-MDB search algorithms that surpass the trivial time complexity of $O^*(2^n)$.

3.1 A New Branch-and-bound Algorithm

We firstly introduce a branch-and-bound algorithm for the k-MDB problem. It employs a straightforward branch-and-bound approach by selecting a vertex v from the graph and dividing the k-MDB search problem into two subproblems: one seeks a k-MDB that includes v, while the other seeks a k-MDB that excludes v. Specifically, given a bipartite graph G, denote by $S = (U_S, V_S)$ a partial set and $C = (U_C, V_C)$ a candidate set, where $G(S)$ is a k-defective biclique of G, and for any vertex $u \in C$, $G(S \cup \{u\})$ forms a larger k-defective biclique of G. (S, C) is referred to as an instance, which aims to identify a k-MDB consisting of S and a subset of C. In particular, the instance $((\emptyset, \emptyset), (U, V))$ aims to find a k-MDB of G, while the instance $(S^*, (\emptyset, \emptyset))$ corresponds to a possible solution $G(S^*)$. To clarify, we refer to a vertex $u \in C$ as a *branching vertex* when a new sub-instance is generated by expanding S with u.

Given an instance (S, C), our branch-and-bound algorithm selects a branching vertex $u \in C$ and generates two new sub-instances: $(S \cup \{u\}, C \setminus \{u\})$ and $(S, C \setminus \{u\})$. The algorithm then recursively processes each sub-instance in the same manner until the candidate set becomes empty, where the partial set yields a possible solution. Finally, the k-MDB of G is obtained as the yielded solution with most edges. The simplest approach to obtain a branching vertex is to randomly select one from C. However, this strategy may lead to the selection of all vertices in C, leading to a worst-case time complexity of $O^*(2^n)$ and too many unnecessary computations. To improve the efficiency and reduce the time complexity, we propose a new binary branching strategy.



(a) the input graph (b) the branching tree

Figure 2: An example of the new binary branching strategy, where $k = 2$ and current instance is $S = (\{u_2\}, \{v_2\})$ (green vertices) and $C = (\{u_1, u_3, u_4, u_5\}, \{v_1, v_3, v_4\})$ (uncolored vertices).

Intuitively, if k vertices from C with non-neighbors in S are successively added to S, then S will contain at least k non-edges. Consequently, this implies that all vertices in C must be the neighbors of each vertex in S. Additionally, if no vertex in C that has non-neighbors in S, adding a vertex v from C to S will result in exactly $\overline{d}_C(v)$ vertices in C with non-neighbors in S. Based on these observations, we give our new binary branching strategy as follows.

New binary branching strategy. Given an instance (S, C), let $u = \arg\max_{u \in C} \overline{d}_S(u)$. The sub-instances are generated in the following way:

- If $\overline{d}_S(u) > 0$, select u as the branching vertex and generate two sub-instances: $(S \cup \{u\}, C \setminus \{u\})$, $(S, C \setminus \{u\})$.
- Otherwise, select $v \in C$ with the maximum $\overline{d}_C(v)$ as the branching vertex, and generate two sub-instances: $(S \cup \{v\}, C \setminus \{v\})$, $(S, C \setminus \{v\})$.

Example 3. Fig. 2 shows an example of our new binary branching strategy for finding a 2-MDB. The input graph is depicted in Fig. 2a, where the current instance is $S = (\{u_2\}, \{v_2\})$ (green vertices) and $C = (\{u_1, u_3, u_4, u_5\}, \{v_1, v_3, v_4\})$ (uncolored vertices). Fig. 2(b) illustrates the branching tree obtained by applying our binary branching strategy. Specifically, in the top-level branch, v_4 is selected as the branching vertex due to its maximum number of non-neighbors in S. When v_4 is added to S, it brings a non-edge in $S \cup \{v_4\}$. Then, we observe that there is no vertex in the second-level branch with non-neighbors in $S \cup \{v_4\}$. Therefore, v_3 is selected as the second-level branching vertex as it has the most non-neighbors in $C \setminus \{v_4\}$. In the third-level branch, u_1 is selected as the branching vertex as it has the most non-neighbors in $S \cup \{v_4, v_3\}$. In the forth-level, $S \cup \{u_1, v_3, v_4\}$ contains 2 non-edges, leading to the deletion of u_3 from C since it still has non-neighbors in $S \cup \{u_1, v_3, v_4\}$, which is the key to reducing the unnecessary computations.

Implementation details. Algorithm 1 outlines the pseudocode of our proposed branch-and-bound algorithm. It takes a bipartite graph G and two integers k and θ as input parameters, and returns a k-MDB D^* of G. It first initializes D^* as an empty graph, and then starts to search k-MDB by invoking the procedure `BranchB` where all vertices in G serve as the candidate set (line 2). `BranchB` holds an instance (S, C) as its parameter, where S denotes the partial set and C denotes the candidate set. Within `BranchB`, the algorithm first determines whether a larger k-defective biclique has been found (lines 5-8). After that, it selects a branching vertex u from C using the new binary branching strategy (lines 9-10) and generates two sub-instance $(S', C' \cup \{u\})$ and $(S, C \setminus \{u\})$ (lines 11-12). Here, S' and C' are the new partial set and new candidate set derived by

Algorithm 1: The branch-and-bound algorithm

Input: Bipartite graph $G = (U, V, E)$, integers k and θ
Output: A k-MDB of G

1 $D^* \leftarrow$ an empty graph;
2 BranchB($(\emptyset, \emptyset)$, (U, V));
3 **return** D^*;
4 **Procedure** BranchB($S = (U_S, V_S), C = (U_C, V_C)$):
5 **if** $C = \emptyset$ **then**
6 **if** $|E_{G(S)}| > |E_{D^*}|$ **and** $|U_S| \geq \theta$ **and** $|V_S| \geq \theta$ **then**
7 $D^* \leftarrow G(S)$;
8 **return**;
9 $u \leftarrow \arg\max_{v \in C} \overline{d}_S(v)$;
10 **if** $\overline{d}_S(u) = 0$ **then** $u \leftarrow \arg\max_{v \in C} \overline{d}_C(v)$;
11 $(S', C') \leftarrow$ Update(S, C, u);
12 BranchB($S' \cup \{u\}, C'$); BranchB($S, C \setminus \{u\}$);
13 **Function** Update(S, C, u):
14 $C' \leftarrow \{v \in C \setminus \{u\} \mid \overline{d}_{S \cup \{u\}}(v) \leq k - \overline{d}(S)\}$;
15 $C'_0 \leftarrow \{v \in C' \mid \overline{d}_{S \cup \{u\} \cup C'}(v) = 0\}$;
16 **return** $(S \cup C'_0, C' \setminus C'_0)$;

invoking Update (line 11), ensuring the candidate set keeps all valid branching vertex (lines 14-15). We next analyze the time complexity of Algorithm 1.

Theorem 4. Given a bipartite graph $G = (U, V, E)$ and an integer k, the time complexity Algorithm 1 is given by $O(m\alpha_k^n)$, where α_k is the largest real root of equation $x^{2k+5} - 2x^{2k+4} + x^3 - x^2 + 1 = 0$. Specifically, when $k = 1, 2$ and 3, we have $\alpha_k = 1.911, 1.979$ and 1.995, respectively.

Proof. Let $T(n)$ denote the maximum number of leave instances generated by BranchB (i.e. instances with an empty C), where $n = |C| = |U_C| + |V_C|$. Since each recursive call of BranchB takes $O(m)$ time, the total time complexity of BranchB is $O(m \cdot T(n))$.

Then, we analyze the size of $T(n)$ based on the binary branching strategy. In each branch of BranchB, either a vertex with at least one non-neighbor in S or a vertex with the most non-neighbors in C is selected as the branching vertex. If the vertex u with at least one non-neighbor in S is selected, it results in at least one additional non-edge in $S \cup \{u\}$. Alternatively, if a vertex v with the most non-neighbors in C is selected as the branching vertex, the next-level branch will contain at least $\overline{d}_C(v)$ vertices in C that have non-neighbors in $S \cup \{v\}$, where $\overline{d}_C(v) > 0$. Consequently, we can conclude that in the worst case, adding two vertices from C to S will result in at least one additional non-edge in S. After adding at most $2k + 1$ vertices from C to S, we have $|\overline{E}_S| = k$ and there exists a vertex $v \in C$ such that $\overline{d}_S(v) > 0$. Based on this, the recurrence is given by $T(n) \leq \sum_{i=1}^{2k+1} T(n - i) + T(n - 2k - 2)$.

Note that the $(2k+2)$-th level of branching (the term $T(n-2k-2)$) corresponds to a branch that searches for the maximum biclique. In this case, it is evident that any vertex in C selected as the branching vertex will result in at least one additional vertex being removed from C. Thus, we yield a recurrence of $T(n - 2k - 2) = T(n - 2k - 3) + T(n - 2k - 4)$ for such a branch. Based on above discussion, we can present the following tighter recurrence.

$$T(n) \leq \sum_{i=1}^{2k+1} T(n - i) + T(n - 2k - 3) + T(n - 2k - 4). \quad (1)$$

According to the theory proposed in [20], $T(n)$ can be expressed in the form of α_k^n, where the value of α_k derives from the largest real root of equation $x^n = x^{n-1} + x^{n-2} + \cdots + x^{n-2k-1} + x^{n-2k-3} + x^{n-2k-4}$, which can be simplified to $x^{2k+5} - 2x^{2k+4} + x^3 - x^2 + 1 = 0$. Therefore, the theorem is proven. □

Lemma 2. Given a bipartite graph $G = (U, V, E)$, the time complexity of Algorithm 1 for finding a 0-MDB of G is $O(m \times 1.618^n)$.

Proof. When $k = 0$, the vertex $v \in C$ with maximum $\overline{d}_C(v)$ is selected as the branching vertex. Moving v from C to S results in the deletion of v's non-neighbors in C, leading to the recurrence $T(n) \leq T(n - 1) + T(n - 2)$, which yields $T(n) = 1.618^n$. □

3.2 A Novel Pivoting-based Algorithm

We observe that Algorithm 1 may still generate numerous unnecessary sub-instances. Consider an instance (S, C) and a branching vertex $v \in C$ with $\overline{d}_S(v) = 0$. In Algorithm 1, if v and all non-neighbors of v have been excluded from the current instance, the remaining vertices cannot form any k-MDB. This is because any k-defective biclique derived from these remaining vertices can always be expanded by including v. Consecutively, Algorithm 1 may produce many k-defective bicliques that are not maximum. To address the problem and further improve the efficiency, we propose a novel pivoting-based algorithm in this section. We formalize above insights into the following lemma.

Lemma 3. Given an instance (S, C) and any vertex $v \in C$ with $\overline{d}_S(v) = 0$, the k-MDB of instance (S, C) must contain either v or a vertex in $\overline{N}_C(v)$.

Based on Lemma 3, we develop a pivoting strategy that provides a new way to generate sub-instances. The details are presented as follows.

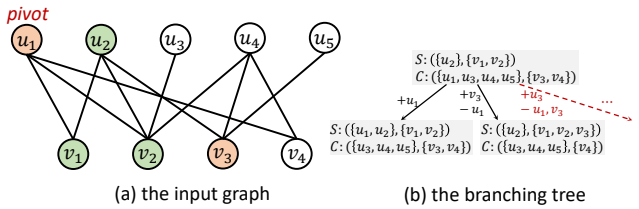
Pivoting strategy. Consider an instance (S, C) and a vertex $u \in C$ with $\overline{d}_S(u) = 0$. Let u be the *pivot vertex*. Assume $\overline{N}_C(u) = \{v_1, v_2, \ldots, v_r\}$, where $r = \overline{d}_C(u)$. The pivoting strategy treats each vertex in $\{u\} \cup \overline{N}_C(u)$ as a branching vertex and generates exactly $r + 1$ sub-instances. The first sub-instance is given by $(S \cup \{u\}, C \setminus \{u\})$, and for $i \geq 2$, the i-th sub-instance is given by $(S \cup \{v_{i-1}\}, C \setminus \{u, v_1, v_2, \ldots, v_{i-1}\})$.

With this strategy, S is expanded only with the vertex u and vertices in $\overline{N}_C(u)$, avoiding redundant sub-instances that would lead to non-maximum k-defective biclique. Next, we consider the selection of the pivot vertex. Let $C_0 \subseteq C$ be the set of vertices fully connected to S, i.e., $C_0 = \{v \in C \mid \overline{d}_S(v) = 0\}$. According to the pivoting strategy, any vertex from C_0 can serve as a pivot vertex. To minimize the number of sub-instances, the vertex $u \in C_0$ with the fewest non-neighbors in C is chosen as the pivot vertex.

In the case where C_0 is empty, every vertex in C has at least one non-neighbor in S, making the pivoting strategy inapplicable. Fortunately, as discussed in Sec. 3.1, continuously branching k times by adding vertices from C to S will result in S containing at least k non-edges. Moreover, C becomes an empty set after this process, which significantly reduces unnecessary computations. Motivated by this observation, we integrate the pivoting strategy with the binary branching strategy to develop a novel pivoting-based branching strategy as outlined below.

Novel pivoting-based branching strategy. Given an instance (S, C), let $C_0 = \{v \in C \mid \overline{d}_S(v) = 0\}$. The new sub-instances of (S, C) are generated in the following way:

- If C_0 is empty or $|C \setminus C_0| > k - |\overline{E}_S|$, let $u = \arg\max_{v \in C} \overline{d}_S(v)$ be the branching vertex and generate two sub-instances: $(S \cup \{u\}, C \setminus \{u\})$ and $(S, C \setminus \{u\})$.
- Otherwise, let $u = \arg\min_{v \in C_0} \overline{d}_C(v)$, and then:



(a) the input graph (b) the branching tree

Figure 3: An example of the pivoting-based branching strategy, where $k = 2$ and current instance is set as $S = (\{u_2\}, \{v_2\})$ (green vertices) and $C = (\{u_1, u_3, u_4, u_5\}, \{v_1, v_3, v_4\})$ (orange and uncolored vertices).

- If $\overline{d}_C(u) > k - |\overline{E}_S| > 0$, generate two sub-instances with u as the branching vertex: $(S \cup \{u\}, C \setminus \{u\})$ and $(S, C \setminus \{u\})$.
- Otherwise, assume $\overline{N}_C(u) = \{v_1, v_2, \ldots, v_r\}$, where $r = \overline{d}_C(v)$, and generate $\overline{d}_C(u) + 1$ sub-instances with u as the pivot vertex, where the first sub-instance is given by $(S \cup \{u\}, C \setminus \{u\})$, and for $i \geq 2$, the i-th sub-instance is given by $(S \cup \{v_{i-1}\}, C \setminus \{u, v_1, v_2, \ldots, v_{i-1}\})$.

Example 5. Fig. 3 shows an example of the our pivoting-based branching strategy for the 2-MDB search. The example graph is given by Fig. 3a, with the current instance set as $S = (\{u_2\}, \{v_1, v_2\})$ (green vertices) and $C = (\{u_1, u_3, u_4, u_5\}, \{v_3, v_4\})$ (orange and uncolored vertices). According to the pivoting-based branching strategy, $C_0 = (\{u_1\}, \{v_3\})$, and u_1 is selected as the pivot vertex since it has the fewest non-neighbors in C among vertices in C_0. Then, only u_1 and all non-neighbors of u_1 in C are used to generate new sub-instances (orange vertices), while other vertices of C are not considered (uncolored vertices). This results in only two sub-instances generated by our pivot-based branching strategy. The branching tree for this example is illustrated in Fig. 3b, where the black arrows represent the generated branches while the red arrows indicate the unnecessary branches.

Implementation details. Algorithm 2 outlines the pseudocode of the proposed pivot-based algorithm. It begins by searching for k-MDB by invoking the procedure BranchP with an empty partial set S and the entire vertex set of G as the candidate set C (line 2). The algorithm returns the global variable D^* as the final k-MDB after BranchP is processed (line 3). Within BranchP, the algorithm first updates D^* if a larger k-defective biclique is found (lines 5-7). Then, it generates new sub-instances based on the pivoting-based branching strategy (lines 8-22). Specifically, if $C_0 = \emptyset$ or $|C \setminus C_0| > k - |\overline{E}_S|$, the vertex $u = \arg\max_{v \in C} \overline{d}_S(v)$ is selected as the branching vertex (line 11), and two new sub-instances are generated (line 13). Otherwise, the algorithm assigns $u = \arg\min_{v \in C_0} \overline{d}_C(v)$. If $\overline{d}_C(u) > k - |\overline{E}_S|$, two sub-instances are generated with u as the branching vertex (lines 16-18). Otherwise, $\overline{d}_C(u) + 1$ new sub-instances are generated with u and all u's non-neighbors in C as branching vertex (lines 20-22). When a branching vertex is added to S, the algorithm adopts the Update procedure presented in Algorithm 1 to update the current instance (lines 12, 17 and 21). Below, we analyze the time complexity of Algorithm 2.

Theorem 6. Given a bipartite graph $G = (U, V, E)$ and an integer k, the time complexity of Algorithm 2 is given by $O(m\beta_k^n)$, where β_k is the largest real root of equation $x^{2k+5} - 2x^{2k+4} + x^{k+3} - 2x + 2 = 0$. Specifically, when $k = 1, 2$ and 3, we have $\beta_k = 1.717, 1.856$ and 1.931, respectively.

Algorithm 2: The pivoting-based algorithm

```
Input: Bipartite graph G = (U, V, E), integers k and θ
Output: A k-MDB of G
1  D* ← an empty graph;
2  BranchP((∅, ∅), (U, V));
3  return D*;
4  Procedure BranchP(S = (U_S, V_S), C = (U_C, V_C)):
5      if C = ∅ then
6          if |E_{G(S)}| > |E_{D*}| and |U_S| ≥ θ and |V_S| ≥ θ then
7              D* ← G(S);
8          return;
9      C_0 ← {v ∈ C | d̄_S(v) = 0};
10     if C_0 = ∅ or |C \ C_0| > k − |Ē_S| then
11         u ← arg max_{v∈C} d̄_S(v);
12         (S', C') ← Update(S, C, u);
13         BranchP(S' ∪ {u}, C'); BranchP(S, C \ {u});
14     else
15         u ← arg min_{v∈C_0} d̄_C(v);
16         if d̄_C(u) > k − |Ē_S| > 0 then
17             (S', C') ← Update(S, C, u);
18             BranchP(S' ∪ {u}, C'); BranchP(S, C \ {u});
19         else
20             for v ∈ {u} ∪ N̄_C(u) do
21                 (S', C') ← Update(S, C, v);
22                 BranchP(S' ∪ {v}, C'); C ← C \ {v};
```

Proof. In Algorithm 2, Let $T(n)$ denote the maximum number of leave instances generated by BranchP, where $n = |C| = |U_C| + |V_C|$. It is easy to verify that finding the vertex with most (least) non-neighbors in S (C) takes at most $O(m)$ time. Then, each recursive call of BranchP takes $O(m)$ time, and thus BranchP takes $O(m \cdot T(n))$ time. We next focus on analyzing the size of $T(n)$. Based on the pivoting-based branching strategy, there are three cases that need to be discussed:

(1) If $C_0 = \emptyset$ or $|C \setminus C_0| > k - |\overline{E}(S)|$, BranchP continuously applies the binary branching strategy. After moving at most k vertices from C to S, S contains k non-edges. At this point, at least one vertex in C will be removed as it has non-neighbors in S. Therefore, the recurrence for this case is given by $T(n) \leq \sum_{i=1}^{k+1} T(n-i)$.

(2) If $k - |\overline{E}_S| = 0$, BranchP degenerates into maximum biclique search and continuously applies the pivoting strategy. Let $r = \overline{d}_C(u)$, the recurrence for this case is given by $T(n) \leq T(n-r-1) + \sum_{i=1}^{r} T(n-r-i)$. If $r \leq 1$, we have $T(n) \leq 2T(n-2)$. If $r \geq 2$, let $T(n) = \gamma_r^n$ according to the existing theory [20], where γ_r is the largest real root of equation $x^n = x^{n-r-1} + \sum_{i=1}^{r} x^{n-r-i}$. The value of γ_r is monotonically decreasing with respect to r when $r \geq 2$. By solving the equation we have $\gamma_2 = 1.395 < \sqrt{2}$. As a result, we can always say $T(n) \leq 2T(n-2)$ for this case.

(3) Otherwise, let $u = \arg\min_{v \in C_0} \overline{d}_C(v)$. If $\overline{d}_C(u) \leq k - |\overline{E}(S)|$, BranchP performs the pivoting strategy with u as the pivot vertex. This strategy generates at most $\overline{d}_C(u) + 1 \leq k + 1$ sub-instances. Therefore, the recurrence for this case is given by $T(n) \leq \sum_{i=1}^{k+1} T(n-i)$. Otherwise, BranchP uses u to perform binary branching strategy. In the next-level branching, we have $|C \setminus C_0| > k - |\overline{E}(S \cup \{u\})|$, as $\overline{d}_C(u) > k - |\overline{E}(S)|$. In this case, the sub-branch (the term $T(n-1)$) will continuously perform the binary branching strategy as discussed in case (1). Consequently, the recurrence for this case is given by $T(n) \leq \sum_{i=1}^{k+1} T(n-i) + T(n-2k-2)$. Note that the term $T(n-2k-2)$ degenerates to maximum biclique search, and we can derive that $T(n-2k-2) = 2T(n-2k-4)$ as discussed

in case (2). Therefore, the final recurrence for this case is given by $T(n) \leq \sum_{i=1}^{k+1} T(n-1) + 2T(n-2k-4)$.

By combining the above analysis, the worst-case recurrence is given by

$$T(n) \leq \sum_{i=1}^{k+1} T(n-1) + 2T(n-2k-4). \quad (2)$$

Based on the existing theorem proposed in [20], we denote by $T(n) = \beta_k^n$, where β_k is the largest real root of equation $x^n = x^{n-1} + x^{n-2} + \cdots + x^{n-k-1} + 2x^{n-2k-4}$. The equation is equivalent to $x^{2k+5} - 2x^{2k+4} + x^{k+3} - 2x + 2 = 0$. Consequently, the theorem is proven. □

Lemma 4. Given a bipartite graph $G = (U, V, E)$, the time complexity of Algorithm 2 for finding a 0-MDB of G is $O(m \cdot 1.414^n)$.

Proof. When setting $k = 0$, Algorithm 2 will always execute the pivoting-based branching strategy (lines 20-22), corresponding to a recurrence of $T(n) < 2T(n-2)$ according to the proof of Theorem 6. Therefore, we have $T(n) = 1.414^n$. □

Discussion. It is worth noting that the branch-and-bound and pivoting techniques has been widely applied on many cohesive subgraph models such as the k-defective clique [8, 9, 13], the k-biplex [50, 51] and the biclique [2, 3, 11]. However, our binary branching strategy and pivoting-based branching strategy are significantly different from all these existing solutions.

Compared to the state-of-the-art maximum k-defective clique algorithm in [13], which achieves a non-trivial time complexity by utilizes a condition to restrict the scenarios of pivoting, directly applying this approach to our pivoting-based algorithm would degrade the time complexity to $O^*(2^n)$. In contrast, we employ a more refined pivoting strategy that considers both the number of non-edges between S and C (lines 10-13 in Algorithm 2) and the number of pivot vertex's non-neighbors (lines 15-18 in Algorithm 2), thereby effectively bounding the time complexity.

Compared to the state-of-the-art pivoting-based maximum biplex search algorithm in [50], which achieves a time complexity of $O(m\gamma_k^n)$, with $\gamma_k = 1.754$ when $k = 1$, our algorithm MDBP reaches a better time complexity. This is mainly because the algorithm in [50] only considers prioritizing k non-neighbors of S, which is similar to our binary branching strategy in Sec. 3.1. By contrast, our pivoting-based branching strategy also bounds the case that fewer than k non-neighbors are between S and C, which reduces the size of branching tree and thereby improves the time complexity (see the proof of Theorem 6).

Compared to the maximal biclique algorithm in [2], our pivot vertex selection approach is much different. During the pivoting process, [2] considers the common neighbors, and selects pivot vertex in a heuristic topological order from only one side, which may not be always optimal. In contrast, our pivoting-based branching strategy selects pivot vertex from both side through deterministic condition, which always generates fewest new sub-branches in each recursion.

4 Optimization Techniques

In this section, we propose various optimization techniques, including graph reduction techniques to eliminate redundant vertices, upper-bounding techniques to prune unnecessary instances, and a heuristic approach for identifying an initial large k-defective biclique. These techniques can be easily integrated into our proposed algorithms in Sec. 3, and can significantly enhance their efficiency.

Algorithm 3: CnReduction(G, k, θ)

```
Input: Bipartite graph G = (U, V, E), integers k and θ
Output: A reduced bipartite graph
1  O ← the ascending degree order of U ∪ V;
2  for u ∈ O do
3      for v ∈ N(u) do
4          for w ∈ N(v) do
5              cn(w) ← 0;
6      for v ∈ N(u) do
7          for w ∈ N(v) do
8              cn(w) ← cn(w) + 1;
9      for v ∈ N(u) do
10         N' ← {w ∈ N(v) | cn(w) ≥ θ − k};
11         if |N'| < θ − k then E ← E \ (u, v) ;
12     if d(u) < θ − k then U ← U \ u;
13     for v ∈ N(u) s.t. d(v) < θ − k do V ← V \ v ;
14 return G;
```

4.1 Graph Reduction Techniques

Common-neighbor-based reduction. For two vertices u and v on the same side, let $cn(u, v)$ (or $cn_S(u, v)$) denote the number of common neighbors of u and v in G (or in S). Given a bipartite graph $G = (U, V, E)$ and a k-MDB $D = (U_D, V_D, E_D)$ of G, every edge $(u, v) \in E_D$ satisfies that (1) $d_D(u) \geq \theta - k$, and (2) for all $w \in N_D(v)$, $cn_D(u, w) \geq \theta - k$. By combining the two conditions, we can conclude that for an edge $(u, v) \in E$, if v has fewer than $\theta - k$ neighbors w such that $cn(u, w) \geq \theta - k$, the edge (u, v) can be pruned in G.

Based on this idea, we present Algorithm 3 to prune such edges from G, referred to as CnReduction. The algorithm enumerates all edges (u, v) in ascending order of u's degree (lines 1-2). For each edges (u, v), the algorithm counts common neighbors between u and the neighbors w of v (lines 3-8). If v has fewer than $\theta - k$ neighbors w satisfying $cn(u, w) \geq \theta - k$, the edge (u, v) is pruned from G (lines 9-11). Additionally, if the degree of u or v drop below $\theta - k$ due to the deletion of (u, v), the algorithm will also prune u or v from G (lines 12-13). We give the time complexity of Algorithm 3 in the following lemma.

Lemma 5. Given a bipartite graph $G = (U, V, E)$, the complexity of Algorithm 3 is $O(d \cdot m)$, where $d = \max_{u \in U \cup V} d(u)$.

In practice, Algorithm 3 performs efficiently although its time complexity is $O(d \cdot m)$. This is because in real-world graphs, most vertices have degrees much smaller than d. As a result, the actual runtime of the algorithm often does not reach this worst-case time complexity.

One-non-neighbor reduction. For an instance (S, C) and a vertex u, let C_u denote the subset of C where all vertices are on the same side as u. If a branching vertex has only one non-neighbor in $S \cup C$, removing it from C can lead to the pruning of additional vertices. The following lemma explains this idea in detail.

Theorem 7. Given an instance (S, C) and a vertex $u \in C$ with $\overline{d}_{S \cup C}(u) = 1$, if u is removed from C, all vertices $v \in C_u$ with $\overline{d}_S(v) \geq 1$ can be pruned from C. Moreover, if $d_C(u) = 1$ and w is the unique non-neighbor of u, then all vertices in $\overline{N}_C(w)$ can also be pruned from C.

Proof. Assume D is a k-MDB derived from (S, C) excluding u. For any $v \in C_u$ such that $v \in D$ and $\overline{d}_S(v) \geq 1$, we have $D \setminus \{v\} \cup \{u\}$ is also a k-MDB. In the case where $\overline{d}_C(u) = 1$ and w is the unique non-neighbor of u, if $w \notin D$, then $D \cup \{u\}$ forms a larger k-MDB. Otherwise, for any $v \in \overline{N}_C(w)$ such that $v \in D$, we have $D \setminus \{v\} \cup \{u\}$ is also a k-MDB. □

Ordering-based reduction. Given a bipartite graph $G = (U, V, E)$, let $u_1, u_2, \ldots, u_{|U|}$ be an arbitrary vertex order of U. For each vertex u_i, the ordering-based reduction aims to find a k-MDB D_i including u_i while excluding $u_1, u_2, \ldots, u_{i-1}$. Obviously, the k-MDB of G is the one with most edges among $D_1, D_2, \ldots, D_{|U|}$. Next, we extend Lemma 1 to constrain the vertex distance in D_i, as shown in the following lemma.

Lemma 6. Given a k-defective biclique $D = (U_D, V_D, E_D)$, if $|U_D| > k$ and $|V_D| > k$, the maximum distance between any two vertices of D is 3.

Lemma 6 indicates that when searching for D_i, vertices with distances more than 3 from u_i can be pruned. Let $N^2(u) = \{w \in N(v) \mid v \in N(u)\}$, i.e., the set of u's 2-hop neighbors. Let $N^2_+(u_i) = N^2(u_i) \cap \{u_{i+1}, u_{i+2}, u_{|U|}\}$. According to Lemma 6, we can construct an initial instance (S_i, C_i) to find D_i, where $S_i = (\{u_i\}, \emptyset)$ and $C_i = (N^2_+(u_i), \bigcup_{u_j \in N^2_+(u_i)} N(u_j))$.

In practical implementation, to identify a larger k-defective biclique as early as possible, we traverse the vertices in U in descending order of their degrees. This approach is based on the intuition that vertices with higher degrees are more likely to be present in an MDB. Additionally, the common-neighbor-based reduction techniques can be employed on (S_i, C_i) to prune unnecessary vertices and edges. Specifically, vertices $u \in C_i$ are pruned if $d_{S_i \cup C_i}(u) < \theta - k$, and edges (u, v) are pruned if there are fewer than $\theta - k$ neighbors w of v satisfying $cn(u, w) \geq \theta - k$.

Progressive bounding reduction. The size threshold θ is provided as an input parameter and can be set to a very small value (e.g., $= k+1$), which may limit the effectiveness of our optimization techniques. To address this issue, we utilize the progressive bounding technique inspired from the maximum biclique problem [36] to provide tighter size thresholds for the k-MDB search problem.

Specifically, let θ^t_U and θ^t_V denote the size thresholds in t-th round. The objective of t-th round is to identify a k-MDB $D_t = (U_{D_t}, V_{D_t}, E_{D_t})$ satisfying $|U_{D_t}| \geq \theta^t_U$ and $|V_{D_t}| \geq \theta^t_V$. Let $D^* = \max\{D_1, D_2, \ldots, D_{t-1}\}$ be the currently largest k-defective biclique. The progressive bounding reduction generates θ^t_U and θ^t_V in the following way:

$$\theta^t_U = \max\{\theta, \lfloor \theta^{t-1}_U / 2 \rfloor\}, \tag{3}$$

$$\theta^t_V = \max\{\theta, \lfloor |E_{D^*}| / \theta^{t-1}_U \rfloor\}. \tag{4}$$

Specially, $\theta^0_U = \max_{u \in U} d_U(u) + k$, and the iteration terminates when $\theta^t_U = 0$. The following lemma shows the correctness of the progressive bound reduction.

Lemma 7. Given a bipartite graph G and a k-MDB $D = (U_D, V_D, E_D)$ of G, there always exists an integer t such that $\theta^t_U \leq |U_D|$ and $\theta^t_V \leq |V_D|$.

In many real-world graphs, θ^t_U and θ^t_V are often significantly larger than θ, which helps facilitate more effective graph reduction, such as a smaller reduced graph by performing the common-neighbor-based reduction.

Discussion. Several prior studies have proposed similar graph reduction techniques for other subgraph models [13, 14, 45, 50, 53]. The progressive bounding reduction have been widely applied on these models [45, 50, 53]. In this work, we adapted these techniques to our problem by incorporating the unique properties of k-defective bicliques. However, our proposed common-neighbor-based reduction and one-non-neighbor reduction are technically distinct from all prior approaches.

On the one hand, the common-neighbor-based reduction leverages the constraints on vertex degree and common neighbors in a k-MDB. The reduction based on vertex degree has been widely applied in the form of core [50, 53]. However, this approach results in a very limited reduction in graph size. Additionally, by applying a single constraint on common neighbors of two vertices u and v where $cn(u, v) < \theta - k$, we can conclude that u and v cannot coexist in the same k-MDB. However, maintaining such coexistence relationships requires $O(n^2)$ space, making it quite challenging for practical implementation.

On the other hand, the one-non-neighbor reduction focuses on analyzing vertices in C that have only one non-neighbor in the current subgraph $G(S \cup C)$. A relevant prior technique addresses the maximum k-defective clique search problem [13], where a vertex in C with exactly one non-neighbor in the subgraph can be directly added to S. However, this reduction cannot be directly applied to the k-MDB problem, as such a vertex may not belong to any k-MDB. For example, consider the following 1-MDB search instance (S, C), where $|U_S| = 2$, $|V_S| = 3$, $U_C = \{u\}$ and $V_C = \{v\}$. $G(S)$ has one non-edge, u and v are non-neighbors. Directly adding v to S results in a non-maximum solution, because $G(S \cup \{v\})$ has 7 edges while adding u to S yields a k-defective biclique with 8 edges. In contrast to [13], we focus on the deletion of such vertices rather than their addition, which restricts the pruning scope to vertices on the same side (as discussed in Theorem 7), thereby ensuring the correctness of the searching result.

4.2 Novel Upper-Bounding Techniques

In this subsection, we propose a novel upper-bounding technique that provides effective upper bounds for both vertices and edges. Formally, given an instance (S, C), assume $u_1, u_2, \cdots, u_{|U_C|}$ and $v_1, v_2, \cdots, v_{|V_C|}$ are the vertices of U_C and V_C in ascending order of $\overline{d}_S(\cdot)$, respectively. Let $\overline{d}^i_S(U_C) = \sum_{t=1}^{i} \overline{d}_S(u_i)$ and $\overline{d}^i_S(V_C) = \sum_{t=1}^{i} \overline{d}_S(v_i)$, i.e., the total number of non-neighbors in S among the first i vertices of U_C and V_C. Based on this, the vertex upper bound is presented in the following lemma.

Lemma 8 (Vertex upper bounds). For any k-MDB $D = (U_D, V_D, E_D)$ derived from (S, C), we have $|U_D| \leq |U_S| + i$ and $|V_D| \leq |V_S| + j$, where i and j are the largest values satisfying $\overline{d}^i_S(U_C) \leq k - |\overline{E}_S|$ and $\overline{d}^j_S(V_C) \leq k - |\overline{E}_S|$, respectively.

Proof. For the side of U, it is obvious that i is the maximum number of vertices that can be added from U_C to U_S such that $(U_S \cup \{u_1, u_2, \ldots, u_i\}, V_S)$ remains a valid k-defective biclique. For any k-MDB derived from (S, C) consisting of $(U_S \cup U_{C'}, V_S \cup V_{C'})$, $(U_S \cup U_{C'}, V_S)$ is still a k-defective biclique. Therefore, we have that $|U_{C'}| \leq i$. The proof on the other side is similar. □

Based on Lemma 8, if both sides reach their vertex upper bounds, an edge upper bound can be calculated as $(|U_S| + i) \times (|V_S| + j) - |\overline{E}_S| - \overline{d}^i_S(U_C) - \overline{d}^j_S(V_C)$. However, this upper bound does not account for the coexistence of vertices in U_C and V_C. To solve the issue, we present a tighter edge upper bound in the following lemma.

Lemma 9 (Edge upper bound). For any k-MDB D derived from (S, C), we have that $|E_D| \leq \max_{0 \leq i \leq |U_C|}(|U_S| + i) \times (|V_S| + j) - |\overline{E}_S| - \overline{d}^i_S(U_C) - \overline{d}^j_S(V_C)$, where j is the largest value satisfying $\overline{d}^i_S(U_C) + \overline{d}^j_S(V_C) \leq k - |\overline{E}_S|$.

Proof. Consider a new instance (S, C') reconstructed from (S, C) by fully connecting vertices between $U_{C'}$ and $V_{C'}$, i.e., $\overline{E}_C = \emptyset$. The ordering of vertices in C' with respected to $\overline{d}_S(\cdot)$ remains unchanged. There exists a k-MDB derived from (S, C') composed of

Algorithm 4: UpperBound$((S, C), k)$

Input: Instance $(S = (U_S, V_S), C = (U_C, V_C))$ and integer k
Output: two vertex upper bounds and an edge upper bound

1 $k' \leftarrow k - |\overline{E}_S|$;
2 Let $u_1, u_2, \cdots, u_{|U_C|}$ and $v_1, v_2, \cdots, v_{|V_C|}$ be the vertices of U_C and V_C in ascending order of $\overline{d}_S(\cdot)$;
3 $\overline{d}_U \leftarrow 0; \overline{d}_V \leftarrow 0; c_U \leftarrow 0; c_V \leftarrow 0$;
4 **for** $i \leftarrow 1 \ldots |U_C|$ *s.t.* $\overline{d}_U + \overline{d}_S(u_i) \leq k'$ **do**
5 $\overline{d}_U \leftarrow \overline{d}_U + \overline{d}_S(u_i)$;
6 $c_U \leftarrow c_U + 1$;
7 $e \leftarrow (|U_S| + c_U) \cdot |V_S| - |\overline{E}_S| - \overline{d}_U$;
8 $i \leftarrow c_U$;
9 **for** $j \leftarrow 1 \ldots |V_C|$ *s.t.* $\overline{d}_V + \overline{d}_S(v_i) \leq k'$ **do**
10 **while** $\overline{d}_U + \overline{d}_V + j > k'$ **do**
11 $\overline{d}_U \leftarrow \overline{d}_U - \overline{d}_S(u_i)$;
12 $i \leftarrow i - 1$;
13 $\overline{d}_V \leftarrow \overline{d}_V + \overline{d}_S(u_i)$;
14 $c_V \leftarrow c_V + 1$;
15 $e \leftarrow \max\{e, (|U_S| + i) \cdot (|U_S| + j) - |E_S| - \overline{d}_U - \overline{d}_V\}$;
16 **return** $|U_S| + c_U, |V_S| + c_V, e$;

S, a prefix of $\{u_1, u_2, \ldots, u_{|U_C|}\}$, and a prefix of $\{v_1, v_2, \ldots, v_{|V_C|}\}$ (otherwise, for any vertex not in a prefix, a preceding vertex can always be found to replace it). Moreover, the k-MDB derived from (S, C') have more edges than any k-MDB derived from (S, C). Consequently, Lemma 9 provide a correct edge upper bound. □

Implementation details. Algorithm 4 presents a pseudocode of our upper-bounding techniques. It starts by setting k' as the remaining number of missing edges (line 1), and sorting the vertices of U_C and V_C in ascending order of $\overline{d}_S(\cdot)$ (line 2), which can be efficiently done using a bucket sort in linear time and space. Then, the algorithm calculates the vertex upper bound in U by finding the longest prefix of U_C that can be added to U_S (lines 4-6), where c_U records the prefix length. After that, the algorithm derives the upper bound c_V for vertices in V using the same way, and simultaneously calculates the upper bound e for edges by removing vertices from the end of the prefix of U_C while computing the longest prefix of V_C (lines 9-15). Finally, it returns two upper bounds of both sides as well as an edge upper bound. We present the time complexity of the algorithm in the following lemma.

Lemma 10. The time complexity of Algorithm 4 is $O(n)$.

4.3 A Heuristic Approach

We propose a heuristic approach to efficiently calculate a large k-defective biclique. We observe that in many real-world graphs, one side of the k-MDB with size threshold θ contains exactly θ vertices, while the other side contains much more than θ vertices. Motivated by this, our heuristic approach aims to find a large k-defective biclique with one side has θ vertices. Specifically, given a graph $G = (U, V, E)$ and an integer k, we initially set two vertex subset $U' =$ and $V' = V$. let $u \in U$ be the vertex with largest $d_{V'}(u)$. If $G(U' \cup \{u\}, V')$ has more than k non-edges, we iteratively delete vertices $v \in V$ with largest $\overline{d}_{U' \cup \{u\}}(v)$ until $G(U' \cup \{u\}, V')$ has no more than k non-edges, and then we add u to U'. We repeat this process until $|U| = \theta$, and yield a large k-MDB $G(U', V')$ if $|V| = \theta$. Otherwise, we yield an empty answer. It is easy to verify that this approach can be finished in $O(\theta n + m)$, which is linear as θ is always a small value.

Algorithm 5: MDBP

Input: A bipartite graph $G = (U, V, E)$, integers k, θ
Output: A k-MDB of G

1 $D^* \leftarrow$ Heuristic(G, k, θ);
2 $\theta_U \leftarrow \max_{u \in U} d(u)$;
3 **while** $\theta_U > \theta$ **do**
4 $\theta_V \leftarrow \max\{\theta, \lfloor |D^*|/\theta_U \rfloor\}$; $\theta_U \leftarrow \max\{\theta, \lfloor \theta_U/2 \rfloor\}$;
5 $G' \leftarrow$ **parallel** CnReduction$(G', k, \min\{\theta_U, \theta_V\})$;
6 $\{u_1, u_2, \ldots, u_{|U|}\} \leftarrow$ the descending degree order of U;
7 **parallel for** $i \in 1 \ldots |U|$ **do**
8 $S \leftarrow (\{u_i\}, \emptyset)$; $C \leftarrow (N_+^2(u_i), \bigcup_{v \in N_+^2(u_i)} N(v))$;
9 $C \leftarrow$ Reduce$(C, u_i, \theta_U, \theta_V)$;
10 BranchP$^*(S, C)$; // BranchP equipped with graph reduction techniques, upper-bounding techniques and parallelization
11 **return** D^*;
12 **Function** Reduce$(C = (U_C, V_C), u, \theta_U, \theta_V)$:
13 $U_{C'} \leftarrow \{v \in U_C \mid cn(u, v) \geq \theta_V - k\}$;
14 $V_{C'} \leftarrow \{v \in V_C \mid d(v) \geq \theta_U - k\}$;
15 **return** $(U_{C'}, V_{C'})$

4.4 Parallelization

To further enhance the scalability of our algorithms for handling larger-scale graphs, we implement parallelization strategy based on our proposed algorithms and optimization techniques. The parallelization of our branch-and-bound and pivoting-based algorithms is based on the search tree of the algorithm (as shown in Fig. 2b and Fig. 3b). Specifically, for a given branching strategy, once an instance (S, C) corresponding to a branch is determined, the sub-branches generated by this branch are uniquely defined and mutually independent. Therefore, we parallelize the processing of sub-branches at lower levels of the search tree. Additionally, the parallelization process is controlled by a global counter that tracks the total number of parallel tasks, and when the counter reaches a predefined threshold, the sub-branches generated by each branch are processed sequentially. Additionally, we also perform parallelization on the ordering-based reduction through the traversal of ordered vertices, and on the common-neighbor-based reduction through the loop in lines 2-13 of Algorithm 3.

4.5 Optimized Algorithms

All the proposed optimization techniques can be applied to our branch-and-bound algorithm and pivoting-based algorithm. We refer to the branch-and-bound algorithm and pivoting-based algorithm equipped with all the optimization techniques as MDBB and MDBP, respectively. An implementation of MDBP is presented in Algorithm 5. The algorithm begins by obtaining an initial k-defective biclique (line 1) by performing the heuristic approach. Then it starts to find k-MDB by performing the progressive bounding reduction (lines 2-4). In each iteration, two size thresholds θ_U and θ_V are generated based on Eq. (3) and Eq. (4). Then, the parallelized common-neighbor-based reduction are employed on the input graph G (line 5) by invoking CnReduction (Algorithm 3). the algorithm continues by performing the parallelized ordering-based reduction on the reduced graph (lines 6-10). In detail, it traverses each vertex u of U in descending degree order (lines 7-8), and constructs an initial instance (S, C). After that, the algorithm further reduce the size of C by invoking Reduce (line 9), which utilizes the common-neighbor-based reduction (lines 13-14). Finally, the algorithm runs BranchP* to find k-MDB in (S, C), which is adapted from BranchP in Algorithm 2 by additionally implementing the graph reduction techniques and the upper-bounding techniques (Algorithm 4). By taking the maximum returned solution from all invocations of BranchP*, the algorithm derives a k-MDB D^* at the end and returns D^* as the final answer (line 11). The implementation

Table 1: Datasets used in experiments

Dataset	$\|U\|$	$\|V\|$	$\|E\|$	Δ	mdbk$\overline{d}$
LKML	42,045	337,509	599,858	(31,719, 6,627)	(14, 1)
Team	901,130	34,461	1,366,466	(17, 2,671)	(1, 39)
Mummun	175,214	530,418	1,890,661	(968, 19805)	(10, 3)
Citeu	153,277	731,769	2,338,554	(189,292, 1,264)	(15, 3)
IMDB	303,617	896,302	3,782,463	(1,334, 1,590)	(12, 4)
Enwiki	18,038	2,190,091	4,129,231	(324,254, 1,127)	(228, 1)
Amazon	2,146,057	1,230,915	5,743,258	(12,180, 3,096)	(2, 4)
Twitter	244,537	9,129,669	10,214,177	(640, 18,874)	(41, 1)
Aol	4,811,647	1,632,788	10,741,953	(100,629, 84,530)	(2, 6)
Google	5,998,790	4,443,631	20,592,962	(423, 95,165)	(3, 4)
LiveJournal	3,201,203	7,489,073	112,307,385	(300, 1,053,676)	(35, 14)

of MDBB is quite similar to that of MDBP, which differs only in that replacing BranchP* in Algorithm 5 with BranchB from Algorithm 1 incorporating the optimization techniques used in BranchP*.

5 Experiments

5.1 Experimental Setup

Datasets. We collect 11 real-world bipartite graphs from diverse categories to evaluate the efficiency of our algorithms. These graphs have been widely used as benchmark datasets for evaluating the performance of other bipartite subgraph search problem [16, 36, 50]. The detailed information of these graphs are shown in Table 1, where Δ and $\overline{d}$ represent the maximum degree and average degree of vertices on each side, respectively. All the datasets are available at http://konect.cc/networks.

Algorithms. We implement two proposed algorithm MDBB and MDBP, which are respectively the branch-and-bound algorithm in Algorithm 1 and the pivoting-based algorithm in Algorithm 2 equipped with all our proposed optimization techniques in Sec. 4. We also implement two baseline algorithms, denoted as MDBase and MDCadp for comparative analysis. Specifically, MDBase is a variant of MDBB with a time complexity of $O^*(2^n)$, which is implemented by substituting the binary branching strategy at line 9-10 in Algorithm 1 with the straightforward strategy that randomly selecting a branching vertex u from C. On the other hand, MDCadp is adapted from the state-of-the-art maximal defective clique algorithm Pivot+ [15]. In detail, MDCadp converts the input bipartite graph into a traditional graph by treating all vertices on the same side as fully connected and runs Pivot+ on the converted graph, subsequently extracting a k-MDB from all the maximal defective cliques of the converted graph. Furthermore, both MDBase and MDCadp incorporate the optimization techniques outlined in Sec. 4.1 and 4.3. All algorithms are implemented in C++ and executed on a PC equipped with a 2.2GHz CPU and 128GB RAM running Linux.

Parameter settings. During the evaluations of all algorithms, we vary the value of k from 1 to 6. As outlined in Sec. 2, we focus on finding a connected k-MDB, achieved by setting $\theta > k$. Based on this, we vary the value of θ from $k+1$ to 10 for a specified k.

5.2 Performance Studies

Exp1: Efficiency comparison among different algorithms. We evaluate the efficiency of two proposed algorithms MDBB and MDBP, along with two baseline algorithms MDBase and MDCadp. The results are shown in Table 2, where the column $|E^*|$ represents the number of edges in the k-MDB, and the notion "-" represents a timeout exceeding 3 hours. As observed, MDBP shows superior performance across most input parameters, achieving two order of magnitude faster than MDCadp on most datasets, and two order of magnitude faster than MDBase on most datasets when $k > 3$. Notably, MDBP essentially solves all tests on most of datasets when setting $k \geq 3$, while MDBase and MDCadp can hardly finish any tests

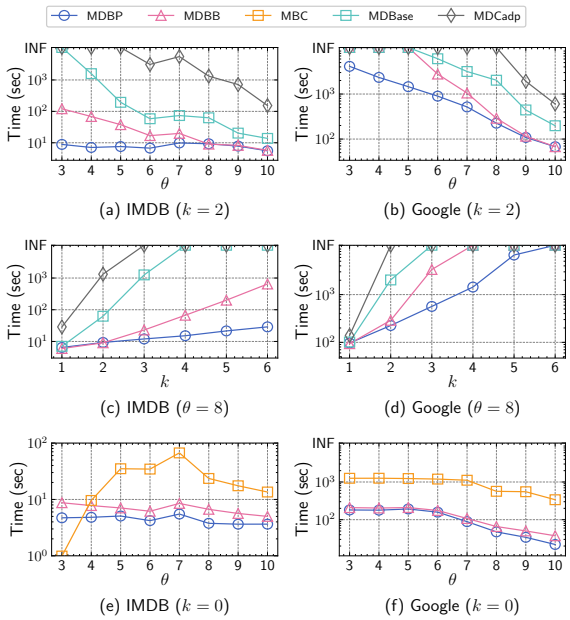


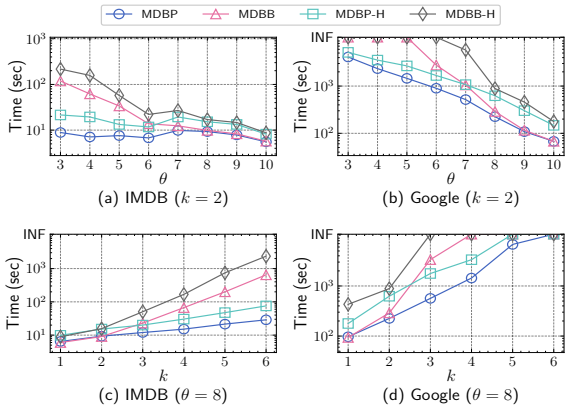
Figure 4: Runtime with varying θ and k

within the time limit. For example, on the *Mummun* dataset with $k = 3$ and $\theta = 7$, MDBase takes 3,677 seconds and MDCadp times out at 108,000 seconds, whereas MDBP completes in just 34 seconds, yielding speedups of 100× over MDBase and 1000× over MDCadp. On the other hand, when $k \leq 2$ and $\theta \geq 5$, the efficiency of MDBP is comparable to MDBB. However, for $k \geq 3$ or $\theta \leq 4$, MDBP demonstrates a significant advantage. For example, on the *Twitter* dataset with $k = 1$ and $\theta = 5$, both MDBP and MDBB runs for about 7 seconds, but with $k = 4$ and $\theta = 5$, MDBB takes about 1 hour while MDBP completes in just 20 seconds, achieving a speedup of more than 150×. These results underscore the effectiveness and high efficiency of the pivoting-based branching strategy mentioned in Sec. 3.2. Additionally, MDBB retains advantages over MDBase and MDCadp, highlighting the effectiveness of the new binary branching strategy in Sec. 3.1.

Exp2: Runtime with varying θ and k. We evaluate the performance of 4 algorithms MDBB, MDBP, MDBase and MDCadp with varying θ and k on two representative datasets, *IMDB* and *Google*. The results are shown in Fig. 4, where the notion "INF" represents the runtime exceeds 3 hours. As observed, both MDBP and MDBB consistently outperform baseline algorithms MDBase and MDCadp with varying θ and k, with MDBP achieving speedups greater than 1000×. For example, in Fig. 4a, MDBP complete in 10 seconds at $\theta = 3$, while MDBase and MDCadp cannot finish within 3 hours. When $\theta \geq 8$ or $k \leq 2$, MDBP and MDBB have similar runtimes, demonstrating the efficacy of our optimization techniques in reducing graph size. Furthermore, as k increases or θ decreases, the time curve of MDBP shows a gentler slope compared to other algorithms, indicating its lower sensitivity to input parameters and superior scalability. Additionally, we evaluate the performance of MDBB and MDBP in searching for the maximum biclique by setting $k = 0$. We compare them with a state-of-the-art maximum biclique search algorithm MBC [36]. The results are shown in Fig. 4e and Fig. 4f. As observed, MDBP and MDBB exhibit an advantage over MBC due to two main factors: (1) both of our algorithms achieves better time complexities than MBC, which has a time complexity of $O^*(2^n)$; and (2) our algorithms incorporate more advanced heuristic approach and graph reduction techniques, further enhancing their efficiency.

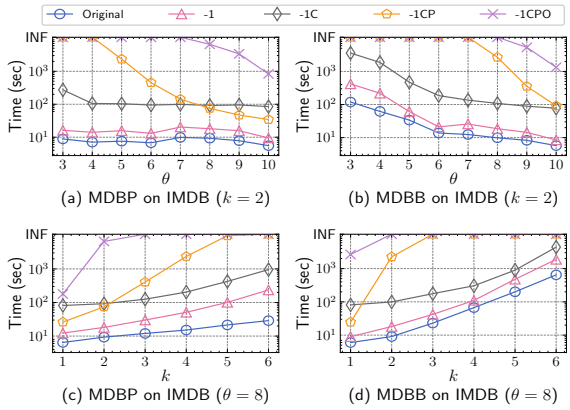
Table 2: Results of runtime among different algorithms on 10 real-world graphs (in seconds)

Dataset	k	θ	$\lvert E^*\rvert$	MDBP	MDBB	MDBase	MDCadp	k	θ	$\lvert E^*\rvert$	MDBP	MDBB	MDBase	MDCadp	k	θ	$\lvert E^*\rvert$	MDBP	MDBB	MDBase	MDCadp
LKML	1	2	1,781	**13.03**	17.08	111.82	-	2	3	460	**43.72**	-	-	-	3	4	149	**267.01**	-	-	-
		5	84	**1.61**	2.15	2.36	28.19		6	70	**7.60**	30.14	357.33	-		7	60	**9.53**	37.96	1,082.91	-
	4	7	66	**103.03**	1,216.70	-	-	5	8	67	**107.31**	1,224.86	-	-	6	9	0	**248.95**	1,732.26	-	-
		8	60	**9.36**	46.38	2,360.74	-		9	0	**13.42**	51.55	3,441.79	-		10	0	**12.29**	46.38	4,162.94	-
Team	1	2	701	**19.26**	39.98	43.67	-	2	3	265	**45.51**	-	-	-	3	4	149	**154.56**	-	-	-
		5	99	**7.14**	7.19	7.73	66.18		6	64	**9.53**	14.09	31.43	-		7	0	**10.07**	15.41	242.20	-
	4	5	111	**591.71**	-	-	-	5	6	79	**2,684.57**	-	-	-	6	8	0	**927.43**	-	-	-
		8	0	**9.92**	15.89	1,734.29	-		9	0	**10.04**	16.35	9,791.32	-		10	0	**7.65**	**11.21**	-	-
Mummun	1	2	10,315	**18.49**	18.62	79.28	-	2	3	8,803	**22.88**	8,661.39	-	-	3	4	4,185	**39.95**	-	-	-
		5	2,469	**13.57**	13.59	16.49	-		6	1,768	**21.08**	37.83	84.27	-		7	1,145	**33.58**	916.16	3,677.02	-
	4	5	2,481	**45.73**	-	-	-	5	6	1,783	**69.39**	-	-	-	6	7	1,163	**160.89**	-	-	-
		8	878	**38.18**	5,538.54	10,440.17	-		9	886	**40.64**	6,737.14	-	-		10	804	**53.09**	-	-	-
Citeu	1	2	12,425	**141.21**	146.81	-	-	2	3	12,427	**148.38**	187.69	-	-	3	4	9,989	**137.15**	4,332.55	-	-
		5	7,394	**5.48**	5.69	-	-		6	4,960	**7.88**	77.20	-	-		7	3,063	**17.92**	3,500.77	-	-
	4	5	7,406	**141.78**	-	-	-	5	6	4,975	**158.43**	-	-	-	6	7	3,081	**253.15**	-	-	-
		8	2,600	**25.92**	-	-	-		9	2,600	**36.03**	-	-	-		10	2,614	**134.21**	-	-	-
IMDB	1	2	775	5.99	**5.39**	4,460.09	-	2	3	562	**8.86**	16.96	-	-	3	4	522	**15.85**	436.35	-	-
		5	514	6.12	**5.35**	145.10	23.43		6	514	**6.76**	6.91	-	1,123.22		7	482	**14.73**	25.23	-	-
	4	5	526	**19.99**	593.86	-	-	5	6	529	**31.05**	683.75	-	-	6	7	498	**57.45**	7,936.24	-	-
		8	436	**15.13**	67.00	-	-		9	436	**19.34**	170.90	-	-		10	426	**22.77**	402.50	-	-
Enwiki	1	2	203,569	**3,630.07**	-	-	-	2	3	133,759	**4,977.99**	-	-	-	3	4	80,461	**3,259.88**	-	-	-
		5	21,939	**156.70**	158.87	354.17	387.15		6	4,240	**199.76**	582.63	-	-		7	3,224	**264.30**	-	-	-
	4	5	21,951	**3,826.16**	-	-	-	5	6	4,255	**5,744.46**	-	-	-	6	8	2,330	**6,557.63**	-	-	-
		8	2,316	**339.60**	-	-	-		9	1,696	**394.81**	-	-	-		10	1,274	**639.63**	-	-	-
Amazon	1	2	1,319	**42.68**	45.42	-	-	2	3	427	**56.54**	646.44	-	-	3	4	423	**88.75**	1,195.66	-	-
		5	413	11.71	**11.12**	12.21	49.70		6	418	12.91	**12.00**	167.48	-		7	357	**13.15**	14.91	183.57	-
	4	5	428	**99.80**	850.49	-	-	5	6	433	**249.97**	-	-	-	6	7	390	**268.21**	-	-	-
		8	368	**13.68**	15.96	171.44	-		9	379	**16.09**	20.38	319.93	-		10	390	**18.63**	25.61	-	-
Twitter	1	2	5,369	**11.13**	13.24	46.13	-	2	3	5,383	**10.88**	51.42	2,143.04	-	3	4	5,397	**15.47**	320.65	-	-
		5	5,369	**6.27**	7.60	36.58	8,976.29		6	5,383	**6.63**	19.51	1,432.31	-		7	5,397	**9.76**	84.34	-	-
	4	5	5,411	**20.75**	3,549.73	-	-	5	6	5,425	**31.20**	-	-	-	6	7	5,439	**57.67**	-	-	-
		8	5,411	**13.29**	491.74	-	-		9	5,425	**23.69**	3,268.33	-	-		10	5,439	**43.93**	9,894.96	-	-
Aol	1	2	10,869	1,032.95	**970.79**	10,736.08	-	2	3	3,208	**1,635.79**	-	-	-	3	4	853	**3,122.56**	-	-	-
		5	334	49.77	**48.69**	49.84	5,703.42		6	238	**94.15**	181.85	581.87	-		7	207	**123.04**	408.97	8,207.80	-
	4	5	346	**9,271.83**	-	-	-	5	7	219	**2,388.21**	-	-	-	6	9	174	-	-	-	-
		8	188	**470.19**	1,011.16	-	-		9	166	**4,496.57**	5,115.84	-	-		10	164	**5,585.25**	8,636.22	-	-
Google	1	2	10,829	**2,163.49**	2,525.77	4,814.91	-	2	3	3,370	**4,121.31**	-	-	-	3	4	1,201	**6,250.09**	-	-	-
		5	439	**552.42**	573.02	629.33	-		6	184	**898.04**	2,771.57	4,077.91	-		7	151	**1,351.14**	-	-	-
	4	7	157	**5,331.22**	-	-	-	5	8	155	**6,750.40**	-	-	-	6	9	156	**8,756.03**	-	-	-
		9	140	**613.17**	9,387.13	-	-		10	145	**683.84**	-	-	-		10	145	**1,978.33**	-	-	-



(a) IMDB ($k = 2$) (b) Google ($k = 2$) (c) IMDB ($\theta = 8$) (d) Google ($\theta = 8$)

Figure 5: Runtime without using the heuristic approach



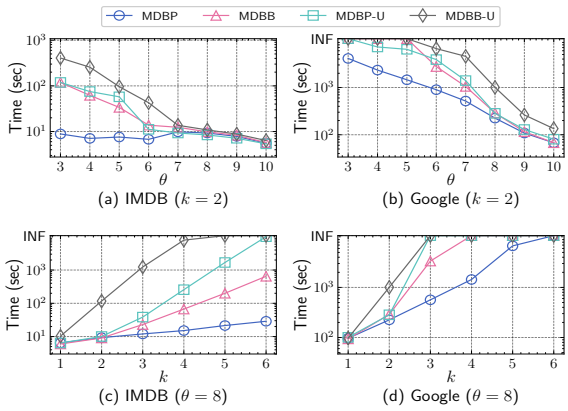
(a) MDBP on IMDB ($k = 2$) (b) MDBB on IMDB ($k = 2$) (c) MDBP on IMDB ($\theta = 8$) (d) MDBB on IMDB ($\theta = 8$)

Figure 6: Runtime without graph reduction techniques

Exp3: Efficacy of the heuristic approach. To evaluate the effectiveness of our proposed heuristic approach, we compare MDBB and MDBP with their versions without the heuristic approach, referred to as MDBB-H and MDBP-H, respectively. The results are shown in Fig. 5. As observed, MDBB and MDBP always outperform MDBB-H and MDBP-H, respectively. This is because the heuristic approach provides a relatively large k-defective biclique at the beginning, which enhances the effectiveness of our upper-bounding techniques. Additionally, we observe that MDBP-H still always performs better than MDBB when $\theta \leq 6$ or $k \geq 3$, which further highlights the high efficiency of our pivoting techniques.

Exp4: Efficacy of graph reduction techniques. To evaluate the effectiveness of our graph reduction techniques, we implement 4 ablation versions of MDBP and MDBB by successively removing the following reductions: one-non-neighbor reduction, common-neighbor-based reduction, progressive bounding reduction and ordering-based reduction. The results are presented in Fig 6, which label the original algorithm as "Original", and label the 4 ablation versions as "-1", "-1C", "-1CP", "-1CPO". As observed, all graph reduction techniques contribute to efficiency improvements. Specifically, the one-non-neighbor and common-neighbor-based reductions yield stable performance improvements with varying θ and k, while the progressive bounding and ordering-based reduction demonstrate more pronounced improvements when θ decreases or k increases. Notably, when $k = 1$ or $\theta = 10$, versions of MDBB and MDBP without progressive bounding reduction still outperform those with it. This suggests that our algorithms are well-designed and robust, achieving strong performance for small k or large θ even without progressive bounding reduction.

Exp5: Efficacy of upper-bounding techniques. To evaluate the effectiveness of our proposed upper-bounding techniques, we compare MDBB and MDBP with their versions excluding the upper-bounding techniques (Algorithm 4), denoted as MDBB-H and MDBP-H, respectively. The results are shown in Fig. 7. As observed, incorporating the upper-bounding techniques significantly improves the efficiency of both MDBB and MDBP. Moreover, the improvement becomes more pronounced as k increases or θ decreases, indicating the strong effectiveness and efficiency of our proposed upper-bounding techniques. Additionally, the improvement of MDBP by performing upper-bounding techniques is more substantial compared to MDBB. This is likely because the pivoting-based branching strategy introduces more non-edges to the partial set S earlier, resulting in tighter vertex and edge upper bounds derived from the upper-bounding techniques, which helps prune more unnecessary instances.



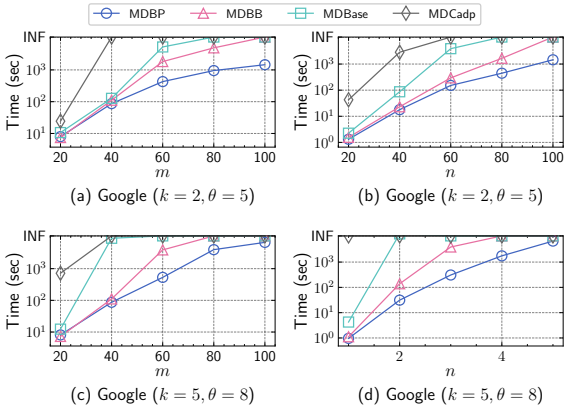
(a) IMDB ($k = 2$) (b) Google ($k = 2$)

(c) IMDB ($\theta = 8$) (d) Google ($\theta = 8$)

Figure 7: Runtime without upper-bounding techniques

Table 3: Runtime on synthetic graphs (in seconds)

Distribution	ρ	Δ	$\overline{d}$	$k=1,\theta=2$			$k=3,\theta=4$			$k=5,\theta=6$		
				$\lvert E^*\rvert$	MDBP	MDBB	$\lvert E^*\rvert$	MDBP	MDBB	$\lvert E^*\rvert$	MDBP	MDBB
Power-law	0.2	(97, 82)	(19, 19)	197	**0.10**	0.18	181	**0.29**	0.96	175	**8.03**	11.82
	0.3	(77, 99)	(29, 29)	269	**0.11**	0.33	277	**0.56**	9.52	292	**16.02**	133.29
	0.4	(95, 100)	(36, 36)	349	**0.27**	**0.51**	357	**1.76**	26.01	375	**33.84**	432.58
	0.5	(100, 99)	(45, 45)	482	**0.92**	**1.94**	494	**5.83**	105.49	506	**189.92**	2,030.16
Normal	0.2	(29, 29)	(17, 17)	27	**0.26**	0.47	29	**31.74**	75.84	32	**59.74**	153.87
	0.3	(43, 43)	(29, 29)	44	**1.24**	3.04	45	**179.49**	397.23	49	**312.55**	1,225.26
	0.4	(48, 51)	(39, 39)	56	**3.78**	10.73	67	**489.87**	1,015.43	65	**1,300.01**	-
	0.5	(62, 61)	(49, 49)	65	**15.03**	43.81	65	**2,095.58**	-	-	-	-



(a) Google ($k = 2, \theta = 5$) (b) Google ($k = 2, \theta = 5$)

(c) Google ($k = 5, \theta = 8$) (d) Google ($k = 5, \theta = 8$)

Figure 8: Scalability of different algorithms

Exp6: Performance on synthetic graphs. To discover the relationship between the efficiency of our approaches and graph characteristics, we evaluate the runtime of MDBB and MDBP on synthetic graphs with varying vertex degree distributions and graph densities. Specifically, we generate 8 synthetic graphs with 100 vertices on each side. Their vertex degrees follows a power-law or normal distribution, and the density $\rho = \frac{|E|}{|U|\cdot|V|}$ varies from 0.2 to 0.5. The results are presented in Table 3. As observed, our algorithms consistently exhibits high efficiency on power-law distribution graphs, whereas their performance on normal distribution graphs is more significantly influenced by ρ and k. This phenomenon can be attributed to two key factors. First, Power-law graphs exhibit a larger gap between Δ and $\overline{d}$ than normal graphs, which facilitates our branching strategies to eliminate more redundant branches and enables our graph reduction techniques to exclude more vertices. This explains why our approaches are highly efficient on large-scale real-world graphs with a significantly gap between Δ and $\overline{d}$, as illustrated in Table 1 and 2. Second, the higher number of k-MDB edges in power-law graphs benefits our upper bound techniques for effectively pruning suboptimal branches, which can also reflected in the real-world graphs in Table 2. Additionally, We observe that MDBP outperforms MDBB on these synthetic graphs, indicating the effectiveness advancement of our pivoting-based branching strategy.

Table 4: Runtime of parallelized algorithms (in seconds)

Dataset	k	θ	MDBP with threads			MDBB with threads		
			1	10	20	1	10	20
Google	1	2	2,163.49	425.76	34.69	3,052.03	870.92	305.05
	3	4	6,250.09	1,324.98	802.67	-	-	10,649.47
	5	6	-	-	8,947.28	-	-	-
LiveJournal	1	2	119.61	89.14	47.11	-	-	9,314.33
	3	4	3,996.77	997.90	850.52	-	-	-
	5	6	-	-	10,016.72	-	-	-

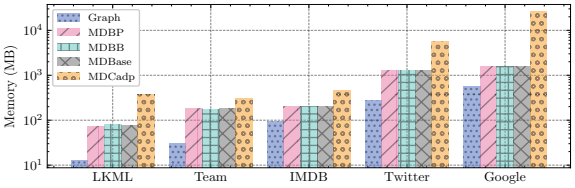
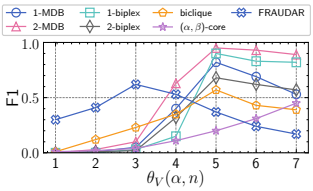


Figure 9: Memory usage of various algorithms

Exp7: Scalability testing. To evaluate the scalability of algorithms, we test the runtime of MDBB, MDBP, MDBase and MDCadp on the *Google* dataset using random sampling of vertices or edges. Results on the other datasets are consistent. We prepare 8 proportionally sampled subgraphs of *Google* by sampling 20%, 40%, 60% and 80% vertices or edges from the original graph. The results are shown in Fig. 8. As observed, MDBP shows the slowest runtime growth as the graph size increases, while MDCadp exhibits the steepest rise; MDBB and MDBase fall in between, where MDBB has better performances. This result indicates that MDBP achieves the best scalability among the algorithms. Furthermore, regardless of graph size, both MDBP and MDBB significantly outperform MDBase and MDCadp, underscoring the effectiveness of the proposed branching strategies and optimization techniques.

Exp8: Efficiency of parallelized algorithms. To evaluate the efficiency of the parallel version of MDBB and MDBP, we test their runtime on two large-scale graphs, *Google* and *LiveJournal*, which are two hard cases for our sequential algorithms as shown in Table 1. The results are shown in Table 4. As observed, the runtime of MDBP and MDBB consistently decreases as the number of threads increases, which illustrates the effectiveness of our parallelization technique. Notably, our parallelized algorithms successfully address two challenging inputs where $k = 5$ and $\theta = 6$ are set on *Google* and *LiveJournal*, which the sequential version of our algorithms failed to solve within 3 hours. This further demonstrates the scalability of our parallelized algorithms.

Exp9: Memory usage. In this experiment, we evaluate the memory usage of algorithms MDBB, MDBP, MDBase and MDCadp on five representative graphs. We measure the maximum physical memory (in MB) used by these algorithms with the parameters $k = 2$ and $\theta = 3$. Similar results can also be observed with other parameters. The results are shown in Fig. 9. As observed, all algorithms except MDCadp exhibit comparable space consumption, which is approximately proportional to the size of input graph. Notably, the space consumed by MDCadp is higher than other algorithms. This is mainly because MDCadp requires to fully connect vertices on both sides of a bipartite graph, resulting in increased graph density and a larger search space. These results indicate that our proposed algorithms are space efficient.



(a) F1-score with varying parameters

Method	F1	Modularity	Time(s)
1-MDB	0.82	0.33	0.30
2-MDB	**0.95**	**0.45**	15.44
1-biplex	0.92	0.37	35.15
2-biplex	0.78	0.24	1187.82
biclique	0.57	0.15	0.23
(α, β)-core	0.45	0.04	0.14
FRAUDAR	0.72	0.27	17.60

(b) Optimal performance

Figure 10: Performance on fraud detection task

5.3 Case Study

We conduct a case study to evaluate the effectiveness of k-MDB in a fraud detection task, following the experimental setup described in [26]. We simulates a camouflage attack scenario on the dataset *Appliance* (https://amazon-reviews-2023.github.io/), which contains 602,777 reviews on 30,459 products by 515,650 users. We inject a fraud block consisting of 500 fake users, 500 fake products, along with 10,000 fake reviews by randomly connect fake users and fake products, and 10,000 camouflage reviews by randomly connect fake users and real products. We adapt MDBP using the approach outlined in [50] to identify the top 2000 k-MDBs on the attacked graph, treating all users and products in these k-MDBs as fake. For comparison, we implement four baseline methods: maximum biclique [36], maximum k-biplex [50] and (α, β)-core[7], where the (α, β)-core is the largest subgraph in which the vertex degrees in left and right side are no less than α and β, respectively. To ensure fairness, we identify the top 2000 largest maximum biclique and maximum k-biplex as fake items. We also incorporate a widely used fraud detection method, FRAUDAR [26], which is a densest-subgraph-based algorithm that identifies suspicious blocks of nodes by iteratively removing edges with the lowest likelihood of being part of a fraudulent structure, thereby resulting in several suspicious subgraphs.

Let θ_U and θ_V denote the size thresholds of users and products, respectively. We use θ_U and θ_V to constrain the side size of k-MDB, maximum k-biplex and maximum biclique. Let n denote the number of blocks that FRAUDAR needs to detect, which is a key parameter that affects the detection accuracy of FRAUDAR. We test the F1-score ($\frac{2\cdot\text{precision}\cdot\text{recall}}{\text{precision}+\text{recall}}$) of each methods with varying θ_V, α and n from 1 to 7, where we fix $\theta_U = \beta = 3$. The results are shown in Fig. 10a. As illustrated, when $\theta_V \geq 5$, 2-MDB consistently outperforms other methods. Additionally, the F1-score of 2-MDB rises over 1-MDB while 2-biplex drops below 1-biplex, as k-MDB offers finer-grained density relaxation than k-biplex with increasing k. For $\theta_V \leq 4$, biclique and FRAUDAR outperform k-MDB and k-biplex, as the latter two are too sparse and skewed to provide meaningful structure. However, the F1-score of biclique declines when $\theta_V \geq 5$, as the strict definition of biclique limits its ability to detect fake items, resulting in lower recall. In contrast, k-MDB and k-biplex, with their more relaxed definitions, still maintain high F1-scores.

We also evaluate the running time and the quality of subgraphs generated by each method, utilizing the widely adopted modularity ($\frac{1}{2m}\sum_{u\in U_S, v\in V_S}(1-\frac{d(u)d(v)}{2m})$) metric, which quantifies the strength of internal connectivity relative to a random distribution and serves as a key indicator for assessing the community structure within the subgraph. Fig. 10b illustrates the optimal performance of each method across all parameter settings. As observed, 2-MDB achieves the highest modularity value, indicating the high quality of the subgraphs it produces, which consequently results in a higher F1 score for fraud detection. Additionally, although 1-biplex has a F1-score close to 2-MDB, its running time is significantly higher than that of 2-MDB. These results demonstrate the high effectiveness and efficiency of our solutions for fraud detection applications.

6 Related Work

Maximum k-defective clique search. As a problem related to our work, maximum k-defective clique search has received considerable attention in recent years, resulting in the development of many solutions and algorithms [8, 9, 12, 13, 21, 24, 27, 35, 42, 44]. Specifically, Trukhanov et al. [44] proposed the first exact algorithm for detecting a maximum k-defective clique with a Russian Doll search scheme. Subsequently, Gschwinda et al. [24] improved the algorithm with a new preprocessing technique and a better verification procedure. These algorithms are derived from more general clique relaxation search frameworks. There are many studies focusing on the maximum k-defective clique. Particularly, Chen et al. [12] proposed a branch-and-bound algorithm with a time complexity below $O^*(2^n)$, and introduced a coloring-based upper bound to further improve the efficiency. Gao et al. [21] proposed another branch-and-bound algorithm with new vertex and edge reduction rules, achieving better practical performance. Chang [8] proposed an algorithm using a non-fully-adjacent-first branching rule, achieving tighter time complexity than the solutions in [12]. Subsequently, Chang [9] further enhanced the algorithm with an ordering technique, resulting in better time complexity and experimental results. Dai et al. [13] proposed a new branch-and-bound algorithm equipped with a pivot-based branching technique and a series of optimization techniques, attaining the best time complexity and experimental results to date. Nevertheless, it is difficult to adapt these algorithms for the k-MDB search problem, which is primarily due to the different definitions of maximum.

Maximum biclique search. The maximum biclique search problem has been extensively studied in recent years [10, 17, 19, 23, 36, 38, 40, 43, 52]. These studies can be classified into three categories. The first category is maximum edge biclique search, which aims to find a biclique with maximum number of edges. Peeters et al. [38] proved the NP-hardness of the problem, and Ambühl et al. [5] showed the inapproximability of the problem. Shaham et al. [40] proposed a probabilistic algorithm for maximum edge biclique search utilizing a Monte-Carlo subspace clustering method. Shahinpour et al. [41] provided an integer programming method with a scale reduction technique. Lyu et al. [36] proposed a branch-and-bound algorithm with a progressive bounding framework and several graph reduction methods, achieving maximum edge biclique search on billion-scale graphs. The second category is maximum vertex biclique search, which aims to identify a biclique with maximum number of vertices. Garey et al. [22] shown that a maximum vertex biclique can be found in polynomial time using a minimum-cut algorithm. The third category is maximum balanced biclique search, which intends to find a maximum biclique with an equal number of vertices on both sides. Yuan et al. [52] proposed an evolutionary algorithm for the problem with structure mutation. Wang et al. [48] proposed a local search framework with four optimization heuristics. Zhou et al. [54] proposed an upper bound propagation served for two algorithms, which are respectively based on branch-and-bound and integer linear programming. Chen et al. [10] introduced two efficient branch-and-bound algorithms utilizing a bipartite sparsity measurement, targeting small dense and large sparse graphs, achieving improved time complexities and better practical performance. All these mentioned techniques are tailored for maximum biclique search and cannot be directly used to solve our k-MDB search problem.

7 Conclusion

In this paper, we investigate the problem of finding a maximum k-defective biclique in a bipartite graph. We propose two novel algorithms to tackle the problem. The first algorithm employs a

newly-designed binary branching strategy that reduces the time complexity to below $O^*(2^n)$, while the second algorithm utilizes a novel pivoting-based branching strategy, achieving a notably lower time complexity. Additionally, we propose a series of non-trivial optimization techniques to further improve the efficiency of our algorithms. Extensive experiments conducted on 10 real-word graphs demonstrate the notable effectiveness and efficiency of our algorithms and optimization techniques.

References

[1] Online. *Maximum Defective Biclique Search in Large Bipartite Graphs (full version).* https://github.com/dawhc/MaximumDefectiveBiclique/blob/main/docs/defbiclique-full.pdf

[2] Aman Abidi, Rui Zhou, Lu Chen, and Chengfei Liu. 2020. Pivot-based Maximal Biclique Enumeration. In *IJCAI*. 3558–3564.

[3] Gabriela Alexe, Sorin Alexe, Yves Crama, Stephan Foldes, Peter L Hammer, and Bruno Simeone. 2004. Consensus algorithms for the generation of all maximal bicliques. *Discrete Applied Mathematics* 145, 1 (2004), 11–21.

[4] Taher Alzahrani and Kathy Horadam. 2019. Finding maximal bicliques in bipartite networks using node similarity. *Applied Network Science* 4, 1 (2019), 21.

[5] Christoph Ambühl, Monaldo Mastrolilli, and Ola Svensson. 2011. Inapproximability results for maximum edge biclique, minimum linear arrangement, and sparsest cut. *SIAM J. Comput.* 40, 2 (2011), 567–596.

[6] Fabian Braun, Olivier Caelen, Evgueni N. Smirnov, Steven Kelk, and Bertrand Lebichot. 2017. Improving Card Fraud Detection Through Suspicious Pattern Discovery. In *Advances in Artificial Intelligence: From Theory to Practice*, Salem Benferhat, Karim Tabia, and Moonis Ali (Eds.). Springer International Publishing, 181–190.

[7] Monika Cerinšek and Vladimir Batagelj. 2015. Generalized two-mode cores. *Social Networks* 42 (2015), 80–87.

[8] Lijun Chang. 2023. Efficient Maximum k-Defective Clique Computation with Improved Time Complexity. *Proceedings of the ACM on Management of Data* 1, 3 (2023), 1–26.

[9] Lijun Chang. 2024. Maximum Defective Clique Computation: Improved Time Complexities and Practical Performance. *arXiv preprint arXiv:2403.07561* (2024).

[10] Lu Chen, Chengfei Liu, Rui Zhou, Jiajie Xu, and Jianxin Li. 2021. Efficient exact algorithms for maximum balanced biclique search in bipartite graphs. In *Proceedings of the 2021 International Conference on Management of Data*. 248–260.

[11] Lu Chen, Chengfei Liu, Rui Zhou, Jiajie Xu, and Jianxin Li. 2022. Efficient maximal biclique enumeration for large sparse bipartite graphs. *Proceedings of the VLDB Endowment* 15, 8 (2022), 1559–1571.

[12] Xiaoyu Chen, Yi Zhou, Jin-Kao Hao, and Mingyu Xiao. 2021. Computing maximum k-defective cliques in massive graphs. *Computers & Operations Research* 127 (2021), 105131.

[13] Qiangqiang Dai, Ronghua Li, Donghang Cui, and Guoren Wang. 2024. Theoretically and Practically Efficient Maximum Defective Clique Search. *Proceedings of the ACM on Management of Data* 2, 4 (2024), 1–27.

[14] Qiangqiang Dai, Rong-Hua Li, Donghang Cui, Meihao Liao, Yu-Xuan Qiu, and Guoren Wang. 2024. Efficient Maximal Biplex Enumerations with Improved Worst-Case Time Guarantee. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–26.

[15] Qiangqiang Dai, Rong-Hua Li, Meihao Liao, and Guoren Wang. 2023. Maximal defective clique enumeration. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–26.

[16] Qiangqiang Dai, Rong-Hua Li, Xiaowei Ye, Meihao Liao, Weipeng Zhang, and Guoren Wang. 2023. Hereditary Cohesive Subgraphs Enumeration on Bipartite Graphs: The Power of Pivot-based Approaches. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 1–26.

[17] Milind Dawande, Pinar Keskinocak, Jayashankar M. Swaminathan, and Sridhar R. Tayur. 2001. On Bipartite and Multipartite Clique Problems. *J. Algorithms* 41, 2 (2001), 388–403.

[18] Kemal Eren, Mehmet Deveci, Onur Küçüktunç, and Ümit V Çatalyürek. 2013. A comparative analysis of biclustering algorithms for gene expression data. *Briefings in bioinformatics* 14, 3 (2013), 279–292.

[19] Qilong Feng, Shaohua Li, Zeyang Zhou, and Jianxin Wang. 2018. Parameterized algorithms for edge biclique and related problems. *Theoretical Computer Science* 734 (2018), 105–118.

[20] Fedor V Fomin and Petteri Kaski. 2013. Exact exponential algorithms. *Commun. ACM* 56, 3 (2013), 80–88.

[21] Jian Gao, Zhenghang Xu, Ruizhi Li, and Minghao Yin. 2022. An exact algorithm with new upper bounds for the maximum k-defective clique problem in massive sparse graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 10174–10183.

[22] Michael R Garey and David S Johnson. 1979. *Computers and intractability*. Vol. 174. freeman San Francisco.

[23] Nicolas Gillis and François Glineur. 2014. A continuous characterization of the maximum-edge biclique problem. *Journal of Global Optimization* 58, 3 (2014), 439–464.

[24] Timo Gschwind, Stefan Irnich, and Isabel Podlinski. 2018. Maximum weight relaxed cliques and Russian Doll Search revisited. *Discrete Applied Mathematics* 234 (2018), 131–138.

[25] Bryan Hooi, Kijung Shin, Hyun Ah Song, Alex Beutel, Neil Shah, and Christos Faloutsos. 2017. Graph-Based Fraud Detection in the Face of Camouflage. *ACM Trans. Knowl. Discov. Data* 11, 4 (2017), Article 44.

[26] Bryan Hooi, Hyun Ah Song, Alex Beutel, Neil Shah, Kijung Shin, and Christos Faloutsos. 2016. Fraudar: Bounding graph fraud in the face of camouflage. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*. 895–904.

[27] Mingming Jin, Jiongzhi Zheng, and Kun He. 2024. KD-Club: An Efficient Exact Algorithm with New Coloring-based Upper Bound for the Maximum k-Defective Clique Problem. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 20735–20742.

[28] MA Langston, Elissa J Chesler, and Y Zhang. 2008. On finding bicliques in bipartite graphs: a novel algorithm with application to the integration of diverse biological data types. In *Proceedings of the 41st Annual Hawaii International Conference on System Sciences (HICSS 2008)(HICSS)*, Vol. 1. 473.

[29] Sune Lehmann, Martin Schwartz, and Lars Kai Hansen. 2008. Biclique communities. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 78, 1 (2008), 016108.

[30] Michael Ley. 2002. The DBLP Computer Science Bibliography: Evolution, Research Issues, Perspectives *(String Processing and Information Retrieval)*. Springer Berlin Heidelberg, 1–10.

[31] Jingdong Li, Zhao Li, Xiaoling Wang, Xingjian Lu, Ji Zhang, and Hongyang Chen. 2023. GPU-Accelerated Maximal Bicliques Mining Framework for Large E-commerce Networks. 539-544 pages.

[32] Jinyan Li, Guimei Liu, Haiquan Li, and Limsoon Wong. 2007. Maximal biclique subgraphs and closed pattern pairs of the adjacency matrix: A one-to-one correspondence and mining algorithms. *IEEE Transactions on Knowledge and Data Engineering* 19, 12 (2007), 1625–1637.

[33] Jinyan Li, Kelvin Sim, Guimei Liu, and Limsoon Wong. 2008. Maximal quasi-bicliques with balanced noise tolerance: Concepts and co-clustering applications. In *Proceedings of the 2008 SIAM International Conference on Data Mining*. SIAM, 72–83.

[34] Guimei Liu, Kelvin Sim, and Jinyan Li. 2006. Efficient mining of large maximal bicliques. In *Data Warehousing and Knowledge Discovery: 8th International Conference, DaWaK 2006, Krakow, Poland, September 4-8, 2006. Proceedings 8*. Springer, 437–448.

[35] Chunyu Luo, Yi Zhou Zhengren Wang, and Mingyu Xiao. 2024. A Faster Branching Algorithm for the Maximum k-Defective Clique Problem. *arXiv preprint arXiv:2407.16588* (2024).

[36] Bingqing Lyu, Lu Qin, Xuemin Lin, Ying Zhang, Zhengping Qian, and Jingren Zhou. 2020. Maximum biclique search at billion scale. *Proceedings of the VLDB Endowment* (2020).

[37] Sara C Madeira and Arlindo L Oliveira. 2004. Biclustering algorithms for biological data analysis: a survey. *IEEE/ACM transactions on computational biology and bioinformatics* 1, 1 (2004), 24–45.

[38] René Peeters. 2003. The maximum edge biclique problem is NP-complete. *Discrete Applied Mathematics* 131, 3 (2003), 651–654.

[39] Amela Prelić, Stefan Bleuler, Philip Zimmermann, Anja Wille, Peter Bühlmann, Wilhelm Gruissem, Lars Hennig, Lothar Thiele, and Eckart Zitzler. 2006. A systematic comparison and evaluation of biclustering methods for gene expression data. *Bioinformatics* 22, 9 (2006), 1122–1129.

[40] Eran Shaham, Honghai Yu, and Xiao-Li Li. 2016. On finding the maximum edge biclique in a bipartite graph: a subspace clustering approach. In *Proceedings of the 2016 SIAM International Conference on Data Mining*. SIAM, 315–323.

[41] Shahram Shahinpour, Shirin Shirvani, Zeynep Ertem, and Sergiy Butenko. 2017. Scale reduction techniques for computing maximum induced bicliques. *Algorithms* 10, 4 (2017), 113.

[42] Vladimir Stozhkov, Austin Buchanan, Sergiy Butenko, and Vladimir Boginski. 2022. Continuous cubic formulations for cluster detection problems in networks. *Mathematical Programming* 196, 1 (2022), 279–307.

[43] Melih Sözdinler and Can Özturan. 2018. Finding maximum edge biclique in bipartite networks by integer programming. In *2018 IEEE International Conference on Computational Science and Engineering (CSE)*. IEEE, 132–137.

[44] Svyatoslav Trukhanov, Chitra Balasubramaniam, Balabhaskar Balasundaram, and Sergiy Butenko. 2013. Algorithms for detecting optimal hereditary structures in graphs, with application to clique relaxations. *Computational Optimization and Applications* 56 (2013), 113–130.

[45] Jianhua Wang, Jianye Yang, Ziyi Ma, Chengyuan Zhang, Shiyu Yang, and Wenjie Zhang. 2023. Efficient Maximal Biclique Enumeration on Large Uncertain Bipartite Graphs. *IEEE Trans. on Knowl. and Data Eng.* 35, 12 (2023), 12634–12648. doi:10.1109/tkde.2023.3272110

[46] Kai Wang, Yiheng Hu, Xuemin Lin, Wenjie Zhang, Lu Qin, and Ying Zhang. 2021. A Cohesive Structure Based Bipartite Graph Analytics System. 4799–4803 pages.

[47] Kai Wang, Wenjie Zhang, Xuemin Lin, Lu Qin, and Alexander Zhou. 2022. Efficient personalized maximum biclique search. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 498–511.

[48] Yiyuan Wang, Shaowei Cai, and Minghao Yin. 2018. New heuristic approaches for maximum balanced biclique problem. *Information Sciences* 432 (2018), 362–375.

[49] Haiyuan Yu, Alberto Paccanaro, Valery Trifonov, and Mark Gerstein. 2006. Predicting interactions in protein networks by completing defective cliques. *Bioinformatics* 22, 7 (2006), 823–829.

[50] Kaiqiang Yu and Cheng Long. 2023. Maximum k-Biplex Search on Bipartite Graphs: A Symmetric-BK Branching Approach. *Proc. ACM Manag. Data* 1, 1 (2023), Article 49.
[51] Kaiqiang Yu, Cheng Long, Shengxin Liu, and Da Yan. 2022. Efficient Algorithms for Maximal k-Biplex Enumeration. In *Proceedings of the 2022 International Conference on Management of Data*. 860–873.
[52] Bo Yuan, Bin Li, Huanhuan Chen, and Xin Yao. 2015. A New Evolutionary Algorithm with Structure Mutation for the Maximum Balanced Biclique Problem. *IEEE Transactions on Cybernetics* 45, 5 (2015), 1054–1067.
[53] Hongru Zhou, Shengxin Liu, and Ruidi Cao. 2025. Efficient Maximum Vertex (k,l)-Biplex Computation on Bipartite Graphs. *Tsinghua Science and Technology* 30, 2 (2025), 569–584. doi:10.26599/TST.2024.9010009
[54] Yi Zhou and Jin-Kao Hao. 2019. Tabu search with graph reduction for finding maximum balanced bicliques in bipartite graphs. *Eng. Appl. Artif. Intell.* 77 (2019), 86–97.